

BÁO CÁO BÀI TẬP TUẦN 5 - BÀI 4 CƠ SỞ LẬP TRÌNH VỚI GEN AI

Học phần: Seminar Chuyên Đề
GVHD: TS. Đỗ Như Tài
Lớp: DCT122C3

Sinh viên thực hiện

STT	Họ và tên	MSSV	Phân công
1	Châu Gia Anh	3122411002	Chương 1, Tổng quan, Side thuyết trình
2	Dương Lê Khánh	3122411093	Chương 2, Lab 2
3	Phan Thành Đại	3122411036	Chương 3, Lab 3
4	Đào Thị Thanh Tâm	3122411182	Chương 4, Lab4 + Chương 5, Lab5

MỤC LỤC

TỔNG QUAN BÁO CÁO.....	3
1. Mục tiêu tài liệu.....	3
2. Giới hạn báo cáo.....	3
3. Phương pháp tiếp cận và triển khai với bài tập.....	3
CHƯƠNG 1: From Automation to Full Software Development Life Cycle: The Current Opportunity for GenAI.....	4
1. Giới thiệu tổng quan.....	4
2. Tiếp cận theo phương pháp luận.....	5
2.1. Mức Ý tưởng (Conceptual Level).....	5
2.2. Mức luận lý (Logic Level).....	6
2.3. Mức vật lý (Physic Level).....	7
3. Tổng kết.....	8
Hình: Sự tăng cường của GEN AI lên vòng đời phát triển phần mềm.....	9
CHƯƠNG 2 : Your Quickstart Guide to OpenAI API.....	10
1. Technical Requirements.....	10
1.1 Environment Setup.....	10
1.2 Required Tools and Libraries.....	10
1.3 Project Resources.....	11
2. Methodology.....	12
2.1 Conceptual Level.....	12
2.1.1 Overview of OpenAI API.....	12
2.1.2 API Communication Model.....	13
2.2 Logical Level.....	14
2.2.1 Obtaining API Keys.....	14
2.2.2 Lab 2.1 – Sending the First OpenAI API Request.....	15
2.2.3 Cost Calculation and Tokens.....	17
2.2.4 Rate Limits and Usage Restrictions.....	17
2.2.5 Lab 2.2 – Analyzing Request Parameters.....	17
2.2.6 Lab 2.3 – Using OpenAI API for Code Generation.....	19
2.3 Physical Level.....	23
2.3.1 Installing OpenAI Python Package.....	23
2.3.2 Example API Request Architecture.....	23
3. Summary.....	23
CHƯƠNG 3: A Guide to GitHub Copilot with PyCharm, VS Code, and Jupyter Notebook.....	25
1. Technical Requirements.....	25
1.1 Environment Setup.....	25

1.2 Required Tools and Libraries.....	25
1.3 Project Resources.....	25
2. Methodology.....	26
2.1 Conceptual Level.....	26
2.1.1 Overview of GitHub Copilot.....	26
2.1.2 Copilot Communication Model.....	26
2.2 Logical Level.....	27
2.2.1 Copilot Setup and Context.....	27
2.2.2 Lab 3.1 – Basic Chat and Code Completion.....	27
2.2.3 Keyboard Shortcuts and Completion Pane.....	28
2.2.4 Lab 3.2 – Advanced Completion for Sequences.....	29
2.2.5 Copilot Edits – Inline modifications.....	29
2.2.6 Lab 3.3 – Code Analysis with Slash Commands Mục tiêu.....	32
2.3 Physical Level.....	36
2.3.1 Installing Copilot Extension.....	36
2.3.2 Example Copilot Architecture in VS Code.....	36
3. Summary.....	36
CHƯƠNG 4: Best Practices for Prompting with ChatGPT.....	38
1. Introduction.....	38
2. Methodology.....	39
2.1. Conceptual Level.....	39
2.2. Logical Level.....	40
2.3. Physical Level.....	40
Lab 4.1 – Converting PyCharm GUI steps to CLI.....	41
Lab 4.2 - Code debugging.....	48
3. Summary.....	53
CHƯƠNG 5: Best Practices for Prompting with OpenAI API and GitHub Copilot.....	54
1. Introduce.....	54
2. Methodology.....	54
2.1. Conceptual Level.....	54
2.2. Logical Level.....	55
2.2.1. Prompting Techniques for the OpenAI API.....	55
2.2.2. Prompting Techniques for GitHub Copilot.....	55
2.3. Physical Level.....	56
Lab 5.1 - Generating docstrings with OpenAI API.....	56
Lab 5.2 - Fixing faulty implementation with GitHub Copilot.....	60
3. Summary.....	69

TỔNG QUAN BÁO CÁO

1. Mục tiêu tài liệu

Báo cáo này nhằm tóm tắt và phân tích phần 1 của cuốn sách "Supercharged Coding with GenAI" (Packt Publishing, 2025), tập trung vào các chương 1 đến 5. Mục tiêu chính là làm rõ vai trò của Trí tuệ Nhân tạo Sinh tạo (GenAI) trong việc nâng cao năng suất lập trình, từ tự động hóa cơ bản đến hỗ trợ toàn bộ chu kỳ phát triển phần mềm (SDLC). Thông qua việc thực hành các lab (Lab 2, 3, 4, 5), nhóm học cách sử dụng các công cụ như OpenAI API, GitHub Copilot và ChatGPT để viết code, debug, và tối ưu hóa prompt. Tài liệu giúp nhóm nắm vững kiến thức lý thuyết, kỹ năng thực hành, và áp dụng GenAI vào lĩnh vực Công nghệ Thông tin, đồng thời ghi nhận đóng góp của từng thành viên theo phân công.

2. Giới hạn báo cáo

Báo cáo chỉ tập trung vào phần 1 của sách (chương 1-5), không mở rộng sang các phần sau hoặc các công cụ GenAI khác ngoài ChatGPT, OpenAI API và GitHub Copilot. Các lab thực hành giới hạn ở Lab 2 (OpenAI API cơ bản), Lab 3 (GitHub Copilot với IDE), Lab 4 (Prompting với ChatGPT), và Lab 5 (Prompting nâng cao với OpenAI API và Copilot), dựa trên mã nguồn từ GitHub. Không bao gồm phân tích sâu về các ứng dụng thực tế ngoài ngữ cảnh học thuật, hoặc so sánh với các mô hình GenAI khác (như Google Bard hoặc Claude). Giới hạn này nhằm đảm bảo tập trung vào yêu cầu bài tập tuần 5.

3. Phương pháp tiếp cận và triển khai với bài tập

Nhóm áp dụng phương pháp luận có cấu trúc với ba mức độ:

1. **Mức Ý tưởng (Conceptual Level):** Tổng quan khái niệm, như vai trò GenAI trong SDLC và cách giao tiếp với các công cụ (ví dụ: mô hình giao tiếp API hoặc Copilot).
2. **Mức Luận lý (Logical Level):** Phân tích logic, bao gồm các bước thực hành lab, tính toán chi phí, giới hạn rate limit, và kỹ thuật prompting (ví dụ: Lab 2.1 gửi request API đầu tiên).
3. **Mức Vật lý (Physical Level):** Triển khai thực tế, như cài đặt package (openai Python), chạy code trên IDE (VS Code, PyCharm), và minh họa bằng screenshot/output.

Triển khai: Nhóm đọc sách, tải mã nguồn từ GitHub, thực hành lab trên môi trường Python, và phân công theo bảng. Toàn bộ quá trình được ghi nhật ký làm việc tại notion và commit log để lưu trữ SHA commit để theo dõi đóng góp.

CHƯƠNG 1: From Automation to Full Software Development Life Cycle: The Current Opportunity for GenAI

1. Giới thiệu tổng quan

Chương 1 của cuốn sách *Supercharged Coding with GenAI* giới thiệu tổng quan về vai trò của Trí Tuệ Nhân Tạo Sinh Tạo (GenAI) trong lĩnh vực lập trình và phát triển phần mềm. Chương này tập trung vào sự chuyển đổi từ các công cụ tự động hóa truyền thống đến việc áp dụng GenAI trong toàn bộ chu kỳ phát triển phần mềm (SDLC - Software Development Life Cycle).

Mục tiêu chính của chương là giúp độc giả hiểu rõ cơ hội mà GenAI mang lại, bao gồm việc tăng tốc độ lập trình, cải thiện chất lượng code, và mở rộng ứng dụng không chỉ ở khâu viết mã mà còn ở các giai đoạn khác như kiểm thử, tài liệu hóa, và bảo trì. Chương nhấn mạnh rằng việc học GenAI đòi hỏi thực hành và thời gian, nhưng nó có thể giúp các lập trình viên từ mới bắt đầu đến chuyên gia nâng cao năng suất đáng kể.

Các chủ đề chính được khám phá bao gồm:

- Sự thay đổi trong lĩnh vực kỹ thuật phần mềm.
- Sự trỗi dậy của các mô hình ngôn ngữ lớn (LLMs).
- Khám phá chu kỳ phát triển phần mềm.
- Áp dụng bộ công cụ GenAI (ChatGPT, OpenAI API, GitHub Copilot).
- Cải thiện phát triển phần mềm với GenAI.
- GenAI có đáng học cho kỹ thuật phần mềm không? (Lợi ích và nhược điểm).
- Các bài học rút ra.

Chương này là nền tảng cho phần còn lại của sách, nhấn mạnh rằng GenAI không chỉ là công cụ hỗ trợ viết code mà còn hỗ trợ toàn diện chu kỳ phát triển phần mềm (SDLC).

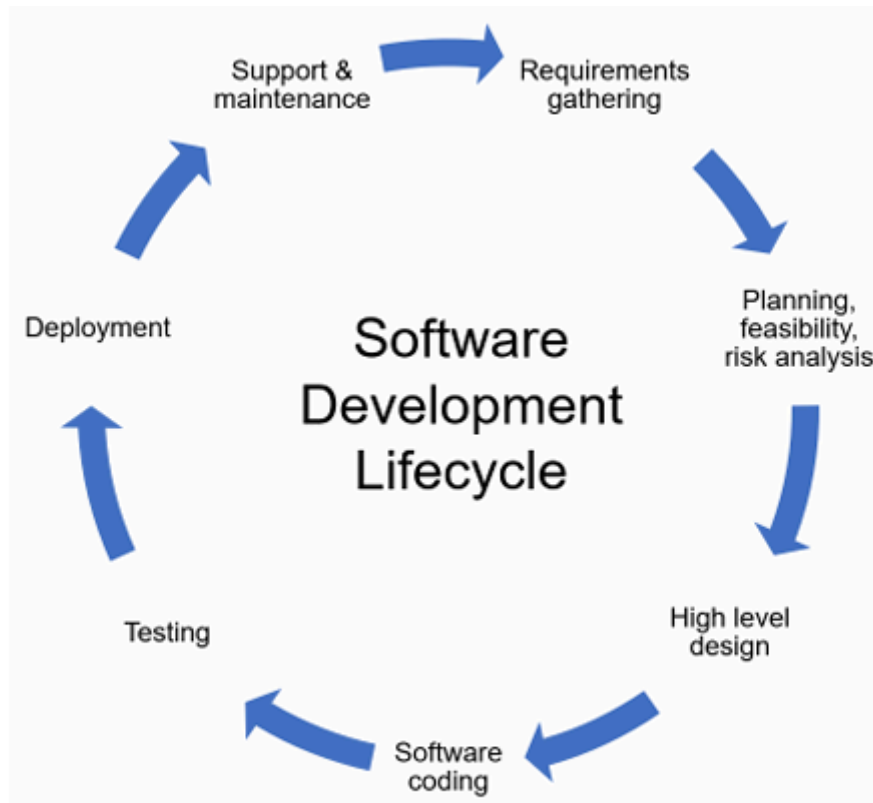


Figure 1.1: The SDLC – the continual process of developing or improving software systems from requirements through maintenance

2. Tiếp cận theo phương pháp luận

2.1. Mức Ý tưởng (Conceptual Level)

Mục này tập trung vào "Tại sao" và "Cái gì" - Tầm nhìn chiến lược của GenAI trong lập trình được thể hiện trong chương.

- **Sự chuyển dịch tư duy:** Khẳng định GenAI không chỉ là một công cụ "gợi ý code" (autocomplete) mà đã trở thành một "Cộng sự lập trình" (AI Pair Programmer).
- **Triết lý "Supercharged Coding":** Tập trung vào việc tăng cường hiệu suất của lập trình viên trong toàn bộ vòng đời phát triển phần mềm (SDLC), không chỉ ở giai đoạn viết code.
- **Vibe Coding:** Một xu hướng mới cho phép phát triển ứng dụng nhanh chóng dựa trên "vibe" (ý tưởng và cảm nhận) thông qua các chỉ dẫn tự nhiên, rút ngắn thời gian từ ý tưởng đến nguyên mẫu (prototype).

2.2. Mức luận lý (Logic Level)

Mục này tổng hợp và mô tả cách thức GenAI vận hành và tích hợp vào quy trình làm việc mà chương đã thể hiện, đồng thời phân tích lợi ích và rủi ro từ việc sử dụng AI để tự động hóa quy trình.

- Sự tiến hóa của công cụ hỗ trợ lập trình
 - Giai đoạn 1: Các trình soạn thảo văn bản thuần túy.
 - Giai đoạn 2: IDE hiện đại với tính năng nhắc mã dựa trên luật (rule-based).
 - Giai đoạn 3 (Hiện tại): LLM (Large Language Models) dựa trên kiến trúc Transformer với khả năng hiểu ngữ cảnh và dự đoán từ tiếp theo dựa trên xác suất.
- Tác động của GenAI lên Vòng đời phát triển phần mềm (SDLC)
 - Lập kế hoạch & Phân tích: Hỗ trợ lên ý tưởng, tóm tắt yêu cầu.
 - Thiết kế & Kiến trúc: Gợi ý các mẫu thiết kế (Design Patterns).
 - Triển khai (Coding): Viết mã nguồn, refactor code cũ.
 - Kiểm thử (Testing): Tự động tạo Unit Test, tạo dữ liệu giả (mock data).
 - Bảo trì: Giải thích mã nguồn phức tạp, viết tài liệu (Docstrings).
- Phân tích Lợi ích và Rủi ro (Trade-offs)
 - Lợi ích: Tăng tốc độ phát triển, giảm bớt các tác vụ lặp lại, hỗ trợ học ngôn ngữ lập trình mới nhanh hơn.
 - Rủi ro: Hiện tượng "ảo giác" (hallucinations) - AI đưa ra thông tin sai nhưng và thuyết phục người dùng tin cũng như vấn đề bảo mật và quyền sở hữu trí tuệ.

Theo nghiên cứu từ AWS, IBM và McKinsey, GenAI thay đổi cấu trúc nỗ lực trong SDLC (Chu kỳ phát triển phần mềm):

- Phân bổ nỗ lực: Lập trình thực tế chỉ chiếm 25-35% tổng thời gian. GenAI tối ưu hóa 65-75% thời gian còn lại thông qua các khâu:
 - Phân tích yêu cầu: Tóm tắt User Story, giảm 60% thời gian xử lý.
 - Lập trình: Giảm 30% thời gian viết mã và tái cấu trúc (refactoring).
 - Kiểm thử: Giảm 25% thời gian tạo unit test; tăng 25% chất lượng mã nguồn.
- Kỹ thuật Prompt Engineering: Là nghệ thuật giao tiếp với mô hình ngôn ngữ lớn (LLM). Một Prompt chuẩn logic bao gồm: Ngữ cảnh (Context) + Chỉ dẫn (Instruction) + Lịch sử (History) + Ví dụ (Examples).

2.3. Mức vật lý (Physic Level)

Phần này đề xuất hiện thực hóa các ý tưởng và luận lý thông qua ba công cụ chủ chốt, tạo thành một hệ sinh thái hỗ trợ lập trình toàn diện. Cuốn sách tập trung vào 3 công cụ vật lý có tính năng riêng biệt:

Công cụ	Bản chất vật lý	Cơ chế hoạt động chính
ChatGPT	AI Chatbot (Web/App)	Tương tác dạng hội thoại (Dialog-driven). Phù hợp để lên ý tưởng và giải thích mã.
OpenAI API	Nền tảng lập trình (Developer Platform)	Cho phép nhúng logic AI (vòng lặp, điều kiện) vào ứng dụng. Phù hợp để tự động hóa quy mô lớn.
GitHub Copilot	IDE Extension (VS Code, PyCharm)	Hoạt động như một Pair Programmer (Lập trình cặp). Dự đoán khối mã tiếp theo dựa trên ngữ cảnh (Context-aware).

1. ChatGPT – Giao diện hội thoại và Kỹ thuật Prompt

- Bản chất: Là một chatbot dựa trên AI, thiết kế cho việc hội thoại bằng ngôn ngữ tự nhiên.
- Đặc điểm kỹ thuật:
 - Sử dụng các phiên bản của mô hình Generative Pretrained Transformer (GPT).
 - Cơ chế tương tác: Dựa trên luồng hội thoại (dialog-driven).
- Phương pháp vận hành: Tập trung vào Kỹ thuật đặt câu lệnh (Prompt Engineering). Một prompt hiệu quả bao gồm: Ngữ cảnh (context), chỉ dẫn (instructions), lịch sử đối thoại (history) và ví dụ (examples).
- Ứng dụng: Sản xuất mã nguồn, viết chú thích (comments), tạo các bản kiểm thử (tests).

2. OpenAI API – Nền tảng lập trình và Tích hợp hệ thống

- Bản chất: Cung cấp nền tảng cho nhà phát triển (Developer Platform) để tương tác trực tiếp với mô hình LLM.
- Sự khác biệt vật lý:

- Cho phép kết hợp giữa lập trình truyền thống (Software Engineering) và kỹ thuật đặt câu lệnh.
- Khả năng kiểm soát luồng: Cho phép sử dụng các cấu trúc logic như vòng lặp (loops) và điều kiện (conditions) – điều mà giao diện chat thông thường thiếu sót.
- Ứng dụng: Giải quyết các bài toán kỹ thuật phần mềm phức tạp ở quy mô lớn, tích hợp trí tuệ nhân tạo vào các ứng dụng riêng biệt.

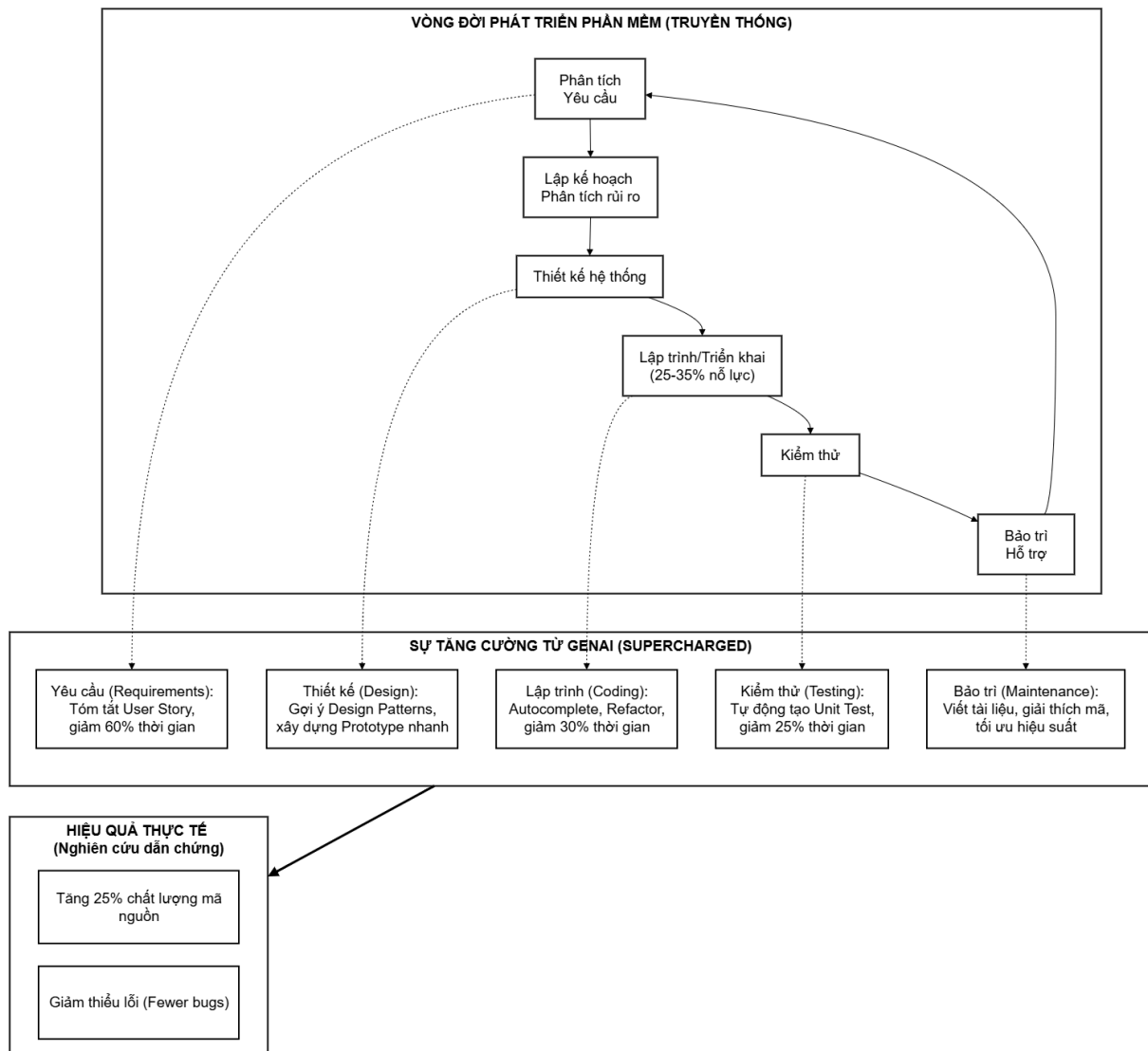
3. GitHub Copilot – Cộng tác lập trình (AI Pair Programmer)

- Bản chất: Công cụ hoàn thiện mã thông minh (Intelligent code completion) tích hợp trực tiếp vào môi trường phát triển (IDE).
- Đặc điểm triển khai:
 - Tích hợp vào các IDE phổ biến: Visual Studio Code và PyCharm.
 - Triết lý vận hành: Dựa trên mô hình Pair Programming (Lập trình cặp) từ phương pháp luận Agile (Extreme Programming - XP).
- Cơ chế dự đoán: Phân tích ý định từ tên hàm, tên biến và sử dụng mã nguồn xung quanh làm ngữ cảnh để dự đoán và gợi ý khối mã tiếp theo.

3. Tổng kết

Bài học tổng kết được rút ra trong chương 1 theo cuốn sách thể hiện quan điểm như sau:

- **Tính hai mặt:** GenAI có thể "Supercharge your strengths" (tăng cường thế mạnh) nhưng cũng có thể "Supercharge your weaknesses" (làm trầm trọng điểm yếu) nếu thiếu sự kiểm chứng (hallucination).
- **Kỹ năng mới:** Lập trình viên hiện đại cần kỹ năng đánh giá rủi ro và chất lượng đầu ra của AI thay vì tin tưởng mù quáng.
- **Mục tiêu:** Sở hữu mã nguồn (Ownership) cho dù nó được viết bởi người hay máy thông qua việc hiểu sâu và kiểm thử chặt chẽ.



Hình: Sự tăng cường của GEN AI bám sát theo vòng đời phát triển phần mềm theo cách tiếp cận truyền thống

CHƯƠNG 2 : Your Quickstart Guide to OpenAI API

1. Technical Requirements

Phần này trình bày các yêu cầu kỹ thuật cần thiết để thiết lập môi trường phát triển và sử dụng OpenAI API.

1.1 Environment Setup

Để thực hiện các ví dụ và bài lab trong chương này, developer cần chuẩn bị các thành phần sau:

- **OpenAI Account:** Người dùng cần tạo tài khoản trên nền tảng OpenAI để có thể truy cập các dịch vụ AI thông qua API.
- **API Key:** API Key được sử dụng để xác thực các request từ ứng dụng đến OpenAI server. API key thường được lưu dưới dạng **environment variable** để đảm bảo tính bảo mật.
- **Python Environment:** Python được sử dụng làm ngôn ngữ lập trình chính để thực hiện các ví dụ và bài thực hành.
- **Integrated Development Environment (IDE):** Một số IDE phổ biến bao gồm:
 - Visual Studio Code
 - PyCharm

IDE giúp developer viết code, chạy chương trình và debug hiệu quả.

1.2 Required Tools and Libraries

Các công cụ và thư viện cần thiết bao gồm:

- Python 3.9 trở lên
- OpenAI Python package
- Virtual Environment

Virtual environment giúp cô lập môi trường project và tránh xung đột thư viện.

Ví dụ tạo môi trường ảo:

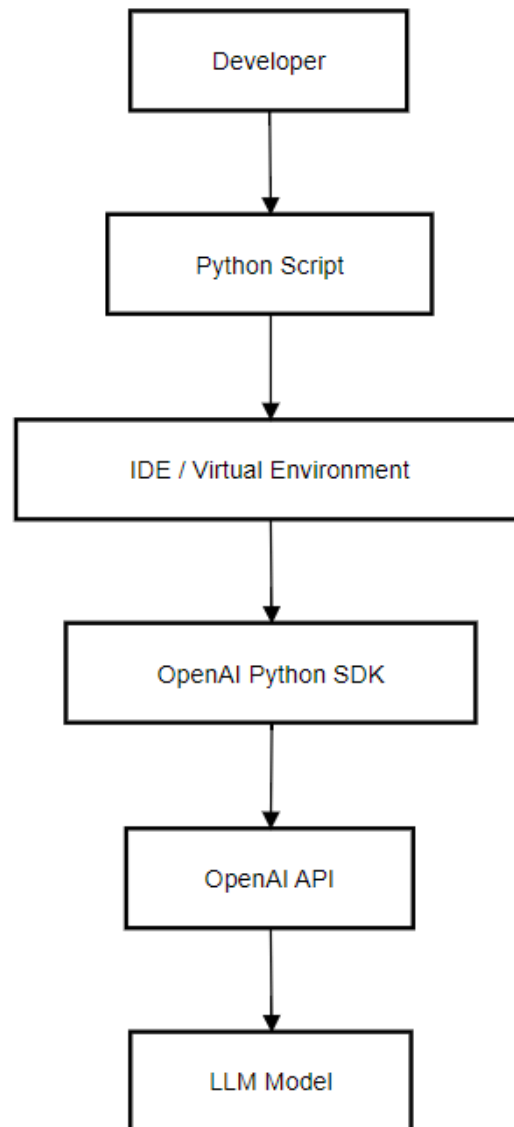
```
python -m venv venv  
source venv/bin/activate
```

1.3 Project Resources

Các tài nguyên hỗ trợ cho chương này bao gồm:

- GitHub repository của sách
- Source code cho các bài lab
- Ví dụ minh họa API usage

Development Environment Architecture:



Giải thích:

1. Developer viết code Python trong IDE.
2. Code chạy trong môi trường ảo.
3. Python script sử dụng OpenAI SDK.
4. SDK gửi request tới OpenAI API.
5. API giao tiếp với mô hình LLM để tạo phản hồi.

2. Methodology

Chương này sử dụng phương pháp tiếp cận ba tầng kiến trúc:

Level	Description
Conceptual	Mô tả nguyên lý hoạt động của OpenAI API
Logical	Mô tả workflow và các lab
Physical	Triển khai bằng code

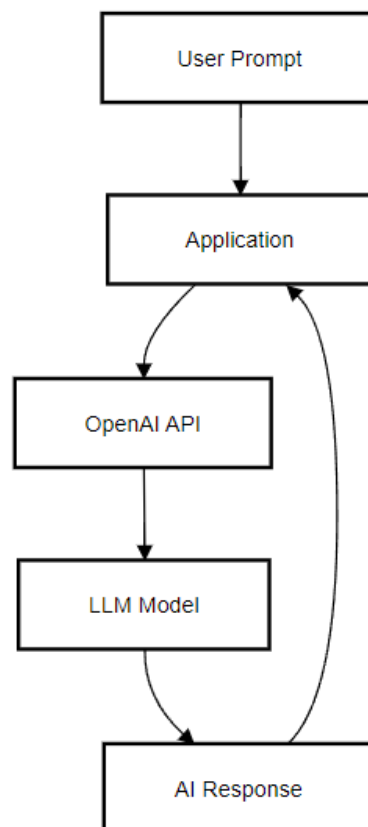
Cách tiếp cận này giúp người học hiểu hệ thống từ ý **tưởng** → **logic** → **triển khai**.

2.1 Conceptual Level

2.1.1 Overview of OpenAI API

OpenAI cung cấp các dịch vụ trí tuệ nhân tạo thông qua Application Programming Interface (API). Developer có thể gửi prompt đến API và nhận phản hồi được tạo bởi Large Language Model (LLM).

OpenAI Interaction Flow:



Quy trình:

1. User nhập prompt.
2. Application gửi request tới OpenAI API.
3. API chuyển dữ liệu tới LLM.
4. LLM xử lý prompt.
5. Kết quả trả về ứng dụng.

2.1.2 API Communication Model

OpenAI API sử dụng **REST architecture** để giao tiếp giữa client và server.

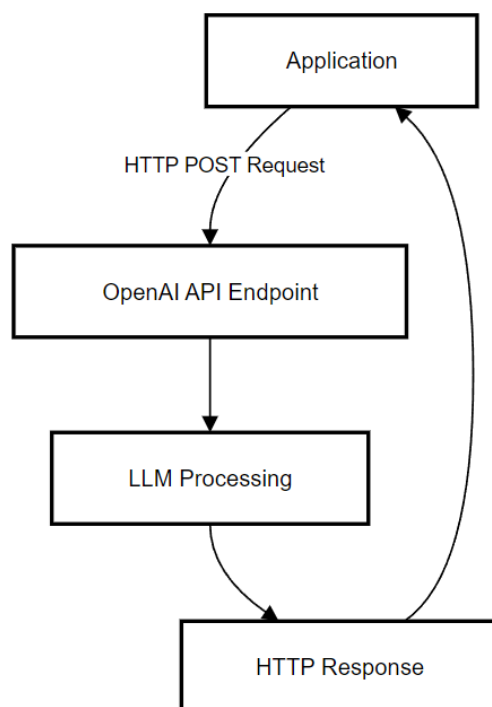
Một API request bao gồm:

- Endpoint
- HTTP method
- Headers
- Body

Ví dụ endpoint:

`POST https://api.openai.com/v1/chat/completions`

API Communication Flow



2.2 Logical Level

Phần này mô tả luồng logic của chương và các bài lab.

2.2.1 Obtaining API Keys

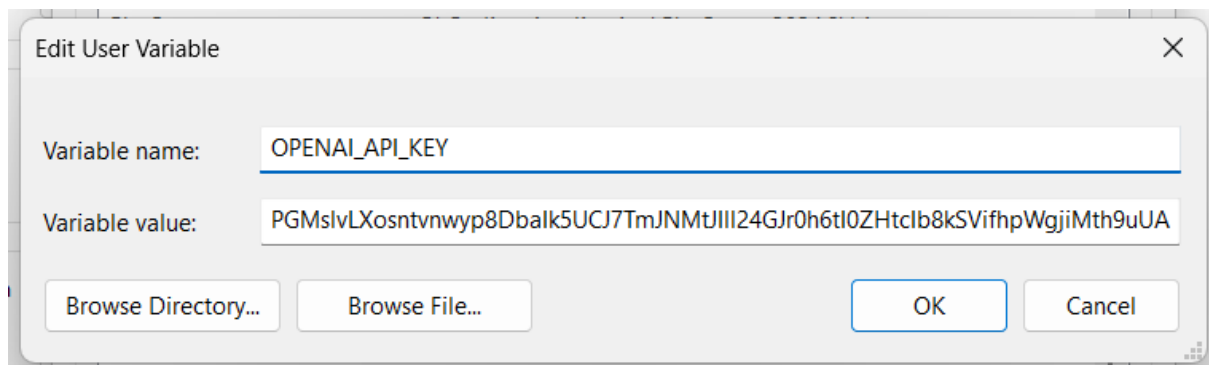
Để sử dụng OpenAI API, developer cần tạo API key từ OpenAI dashboard.

Các bước:

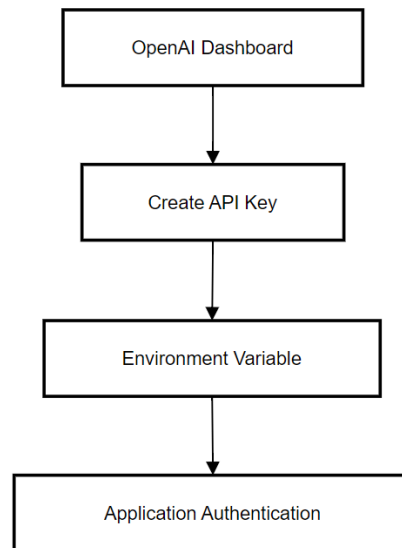
1. Login vào OpenAI platform
2. Tạo API key
3. Lưu API key vào environment variable

Ví dụ:

```
sk-proj-8LEgmior7R1XSgsKN7S85Kg0HNSZZ_qs-xLn33xxxxxxxxxxxxxxxxxxxx  
xxxxxxxxxxxxxxxxxxxx4d5AfFnf7K8-jTL10Y1V6iTQWnhf05mBV_p6PUvrbgHWC  
Yp7Jw9PvpII9UJ88A
```



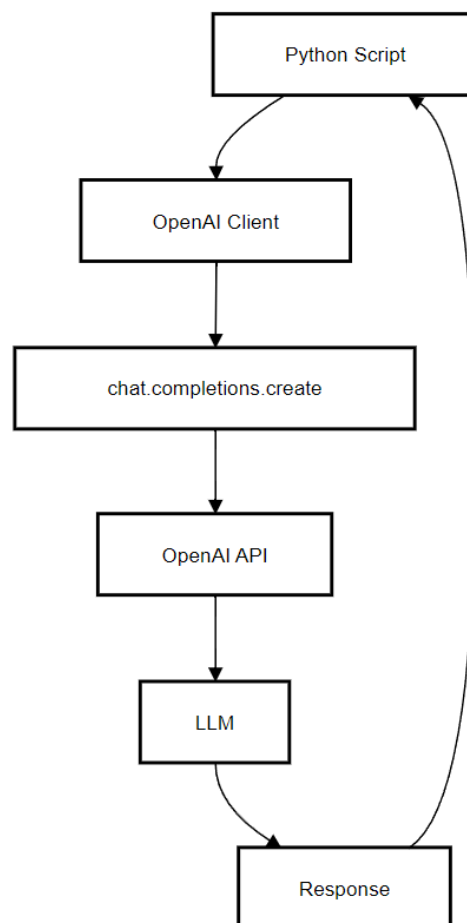
API Key Workflow



2.2.2 Lab 2.1 – Sending the First OpenAI API Request

Mục tiêu của lab này là gửi request đầu tiên tới OpenAI API và nhận phản hồi từ mô hình LLM.

Execution Flow



Code Implementation

```
import openai
from openai import OpenAI

if __name__ == "__main__":

    client: OpenAI = OpenAI()

    completion: openai.ChatCompletion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "What is the FizzBuzz problem?"}]
    )

    print(completion.choices[0].message.content)
```

Code trên thực hiện:

1. Tạo OpenAI client
2. Gửi prompt đến mô hình
3. Nhận phản hồi từ API
4. In kết quả ra màn hình

Program Output

● (openai-lab) PS C:\Users\Lenovo\Downloads\Supercharged-Coding-with-Gen-AI-main\Supercharged-Coding-with-Gen-AI-main\ch2\solutions> py lab21.py

The FizzBuzz problem is a common coding challenge often used in programming interviews to assess a candidate's ability to write simple algorithms. The task is straightforward: Write a program that prints the numbers from 1 to a specified limit (usually 100). However, for multiples of 3, the program should print "Fizz" instead of the number, and for multiples of 5, it should print "Buzz". For numbers that are multiples of both 3 and 5, it should print "FizzBuzz".

Here's a basic outline of the rules:

1. For numbers that are multiples of 3, print "Fizz".
2. For numbers that are multiples of 5, print "Buzz".
3. For numbers that are multiples of both 3 and 5, print "FizzBuzz".
4. For all other numbers, print the number itself.

An example output for the numbers 1 to 15 would look like this:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

...

This problem tests basic programming skills, including loops, conditionals, and logic. It can also serve as a warm-up exercise to help candidates get comfortable with a coding interview format

2.2.3 Cost Calculation and Tokens

OpenAI tính phí dựa trên **tokens**.

Token là đơn vị nhỏ của văn bản được mô hình xử lý.

Thông thường:

1 token $\approx$ 0.75 từ

Chi phí được tính dựa trên:

Cost = Input Tokens + Output Tokens

Developer cần tối ưu prompt để giảm chi phí sử dụng API.

2.2.4 Rate Limits and Usage Restrictions

OpenAI áp dụng các giới hạn sử dụng nhằm đảm bảo hệ thống ổn định.

Các giới hạn chính bao gồm:

- Requests per minute
- Tokens per minute

Ngoài ra còn có sự khác biệt giữa:

- Free accounts
- Paid accounts

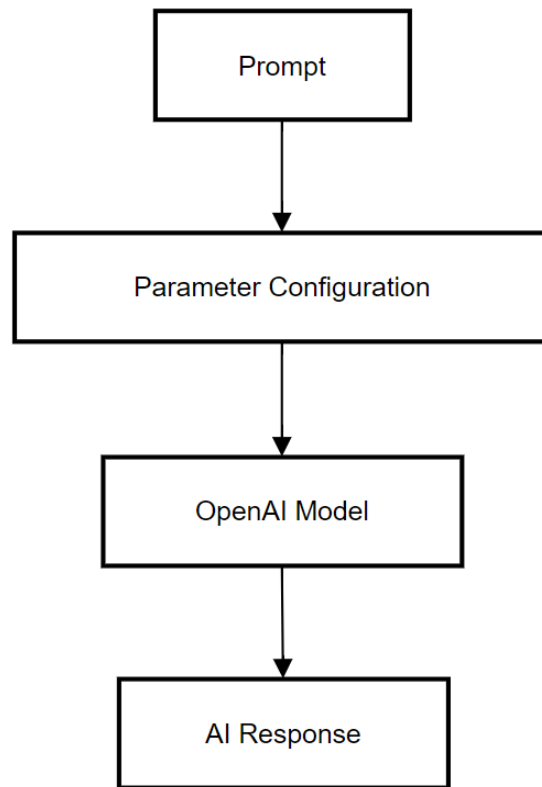
2.2.5 Lab 2.2 – Analyzing Request Parameters

Lab này tập trung vào việc phân tích các request parameters nhằm điều chỉnh hành vi của mô hình.

Các tham số quan trọng:

- temperature
- max_tokens
- n

Parameter Flow



Code Implementation

```
import openai
from openai import OpenAI

if __name__ == "__main__":

    client: OpenAI = OpenAI()

    completion: openai.ChatCompletion = (

        client.chat.completions.create(

            model="gpt-4o-mini",

            temperature=0.2,

            max_tokens=100,
```

```

n=1,

messages=[
    {"role": "system", "content": "You are a hiring manager at a tech
company."},
    {"role": "user", "content": "What is the Two Sum problem?"},
],
)
)

print("Completion Tokens: ", completion.usage.completion_tokens)

print("Output: ", completion.choices[0].message.content)

```

Code trên thực hiện:

- điều chỉnh temperature
- giới hạn max_tokens
- in số token và output của mô hình

Program Output

```

(openai-lab) PS C:\Users\Lenovo\Downloads\Supercharged-Coding-with-Gen-AI-main\Supercharged-Coding-with-Gen-AI-main\ch2\solutions> py lab22.py
Completion Tokens: 100
Output: The Two Sum problem is a classic algorithmic challenge often encountered in coding interviews and computer science courses. The problem can be stated as follows:

Given an array of integers `nums` and an integer `target`, you need to determine if there are two distinct indices `i` and `j` in the array such that the sum of the elements at
these indices equals the target value. In other words, you need to find two numbers in the array that add up to the specified target.

```

2.2.6 Lab 2.3 – Using OpenAI API for Code Generation

Mục tiêu

Lab này minh họa cách sử dụng OpenAI API để tự động sinh code từ mô tả hoặc function signature.

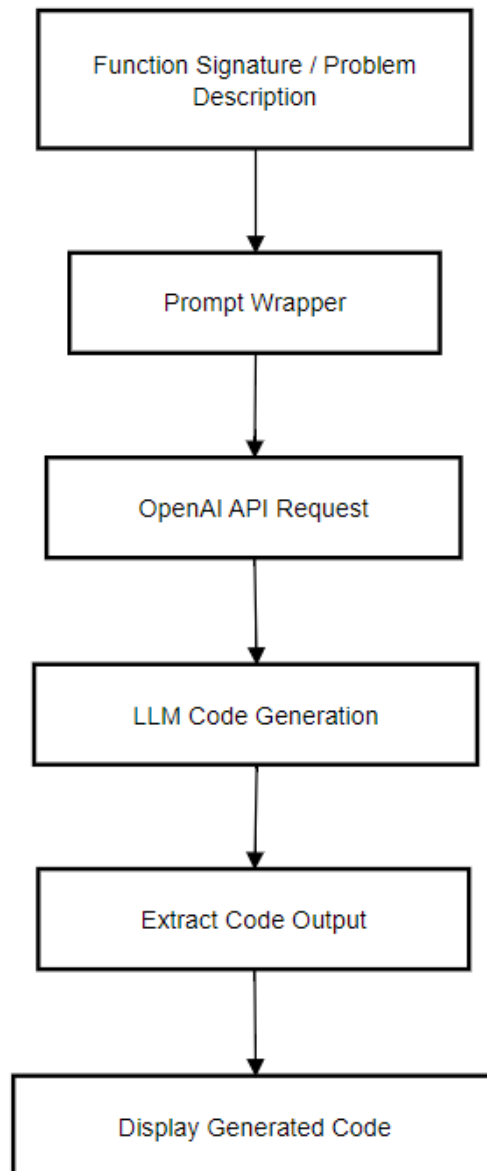
Mô hình LLM được sử dụng như một code assistant, có khả năng:

- hiểu yêu cầu lập trình
- sinh code Python tương ứng
- trả về code dưới dạng văn bản.

Workflow Logic

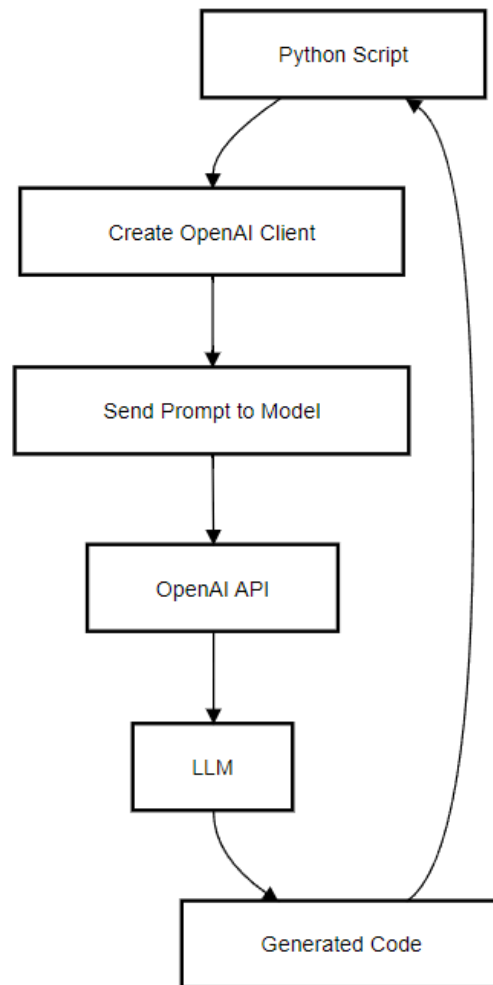
Ở mức logical level, chương trình thực hiện các bước sau:

1. Nhận function signature hoặc mô tả yêu cầu
2. Tạo prompt wrapper
3. Gửi request tới OpenAI API
4. Mô hình LLM sinh code
5. Chương trình trích xuất và hiển thị code



Execution Flow

Diagram chi tiết hơn của luồng thực thi chương trình:



Code Implementation

Code dưới đây gửi yêu cầu tới OpenAI API để sinh code Python.

```
from openai import OpenAI

client = OpenAI()

prompt = """
Write a Python function that solves the Two Sum problem.
"""
```

```
completion = client.chat.completions.create(

    model="gpt-4o-mini",

    messages=[
        {"role": "system", "content": "You are a helpful Python coding assistant."},
        {"role": "user", "content": prompt}
    ]
)

print(completion.choices[0].message.content)
```

Giải thích code

Chương trình thực hiện các bước:

1. Khởi tạo OpenAI client
2. Xây dựng prompt mô tả yêu cầu sinh code
3. Gửi request tới OpenAI API
4. Nhận phản hồi từ mô hình
5. In code được sinh ra

Mô hình sẽ trả về một đoạn code Python giải bài toán Two Sum.

Program Output

```
(openai-lab) PS C:\Users\Lenovo\Downloads\Supercharged-Coding-with-Gen-AI-main\Supercharged-Coding-with-Gen-AI-main\ch2\solutions> py lab23.py
Output 1:
def print_fibonacci_sequence(n: int) -> None:
    """
    Print the Fibonacci sequence up to the n-th number.

    Args:
        n (int): The count of Fibonacci numbers to print.
    """
    a, b = 0, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b
    print() # for final newline
Output 2:
def print_fibonacci_sequence(n: int) -> None:
    """Prints the Fibonacci sequence up to the n-th term.

    Args:
        n (int): The number of terms of the Fibonacci sequence to print.
    """
    if n <= 0:
        return
    elif n == 1:
        print(0)
        return
    elif n == 2:
        print(0)
        print(1)
        return

    a, b = 0, 1
    print(a)
    print(b)

    for _ in range(2, n):
        a, b = b, a + b
        print(b)
```

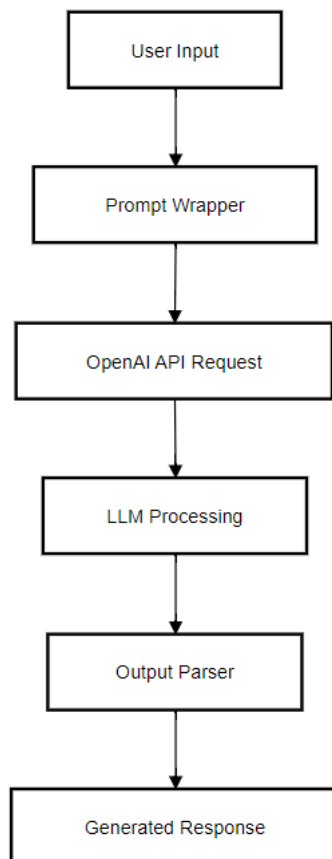
2.3 Physical Level

2.3.1 Installing OpenAI Python Package

Cài đặt thư viện OpenAI:

```
pip install openai
```

2.3.2 Example API Request Architecture



3. Summary

Chương này đã trình bày cách sử dụng OpenAI API từ góc độ kiến trúc đến triển khai thực tế.

Các nội dung chính bao gồm:

- Giới thiệu OpenAI API và mô hình hoạt động của LLM
- Thiết lập môi trường phát triển và API key
- Thực hiện API request đầu tiên
- Phân tích chi phí dựa trên tokens
- Khám phá các tham số điều chỉnh output của mô hình

Thông qua các bài lab, người học có thể hiểu rõ cách tích hợp AI vào ứng dụng thực tế và sử dụng OpenAI API trong lập trình.

CHƯƠNG 3: A Guide to GitHub Copilot with PyCharm, VS Code, and Jupyter Notebook

1. Technical Requirements

Phần này trình bày các yêu cầu kỹ thuật cần thiết để thiết lập môi trường phát triển và sử dụng GitHub Copilot.

1.1 Environment Setup

Để thực hiện các ví dụ và bài lab trong chương này, developer cần chuẩn bị các thành phần sau:

- **GitHub Account & Copilot Subscription:** Người dùng cần có tài khoản GitHub và đăng ký dịch vụ GitHub Copilot để có quyền truy cập vào các tính năng hỗ trợ lập trình bằng AI.
- **Python Environment:** Python được sử dụng làm ngôn ngữ lập trình chính để thực hiện các bài thực hành tính toán trung bình nhân.
- **Integrated Development Environment (IDE):** IDE được sử dụng chính trong phần này là Visual Studio Code (VS Code). IDE giúp developer viết code, thao tác với các phím tắt và gọi các lệnh phân tích mã hiệu quả.

1.2 Required Tools and Libraries

Các công cụ và phần mở rộng cần thiết bao gồm:

- Python 3.9 trở lên.
- GitHub Copilot Extension cài đặt trên VS Code.
- GitHub Copilot Chat Extension (tùy chọn để mở rộng khung chat).

1.3 Project Resources

Các tài nguyên hỗ trợ cho chương này bao gồm:

- Thư mục ch3/labs chứa các file thực hành từ repository của sách.
- Source code cho các bài Lab 3.1, 3.2 và 3.3.

2. Methodology

Chương này sử dụng phương pháp tiếp cận ba tầng kiến trúc:

Level	Description
Conceptual	Mô tả nguyên lý hoạt động của GitHub Copilot và các luồng tương tác AI.
Logical	Mô tả workflow của các tính năng Completion, Chat, Analysis và các bài lab thực hành.
Physical	Triển khai bằng thao tác phím tắt và code thực tế trên VS Code.

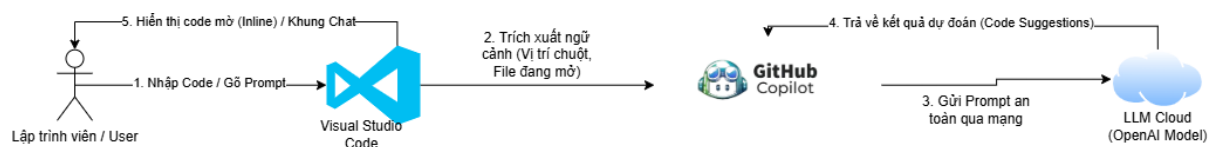
Cách tiếp cận này giúp người học hiểu hệ thống từ ý tưởng → logic → triển khai.

2.1 Conceptual Level

2.1.1 Overview of GitHub Copilot

GitHub Copilot đóng vai trò là một cộng sự lập trình AI (AI Pair Programmer). Thay vì lập trình viên phải tự gõ toàn bộ mã nguồn, phương pháp "Vibe Coding" cho phép con người đóng vai trò kiến trúc sư chỉ đạo logic, còn Copilot sẽ tự động hóa việc sinh mã, gỡ lỗi và phân tích.

GitHub Copilot Interaction Flow:



Quy trình:

- User gõ mã nguồn, bình luận (prompt) hoặc định dạng hàm.
- Copilot Extension trích xuất ngữ cảnh (Context Extraction) từ file đang mở và vị trí con trỏ.
- Dữ liệu được gửi tới LLM tối ưu hóa cho lập trình.
- LLM xử lý và trả về các đoạn mã đề xuất.

2.1.2 Copilot Communication Model

Copilot giao tiếp và hỗ trợ lập trình viên thông qua 3 chế độ chính:

- **Completion (Hoàn thiện mã):** Dự đoán và sinh mã nội tuyến (inline) dưới dạng chữ mờ (ghost text).
- **Chat (Hỏi đáp):** Cung cấp kiến thức nền thông qua giao diện hội thoại tự nhiên.
- **Analysis (Phân tích):** Sử dụng các Slash Commands (/fix, /explain) để thực thi các tác vụ sửa lỗi và giải mã chuyên sâu.

2.2 Logical Level

Phần này mô tả luồng logic của chương và các bài lab.

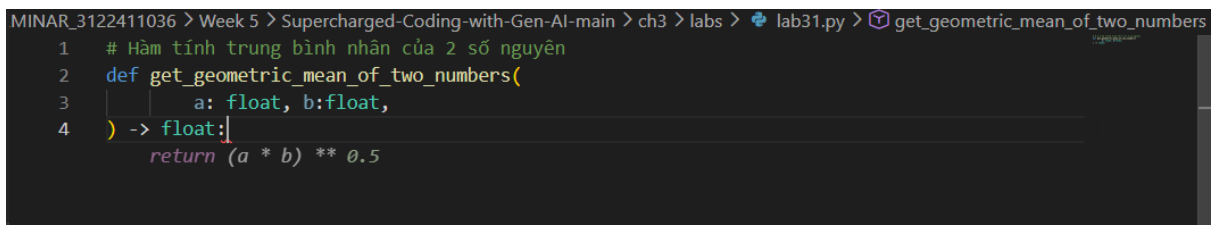
2.2.1 Copilot Setup and Context

Để Copilot hoạt động hiệu quả, developer cần cung cấp ngữ cảnh rõ ràng thông qua tên hàm (function signature) và các kiểu dữ liệu (type hints). Đây được xem là các "prompt" gián tiếp chỉ dẫn cho mô hình LLM.

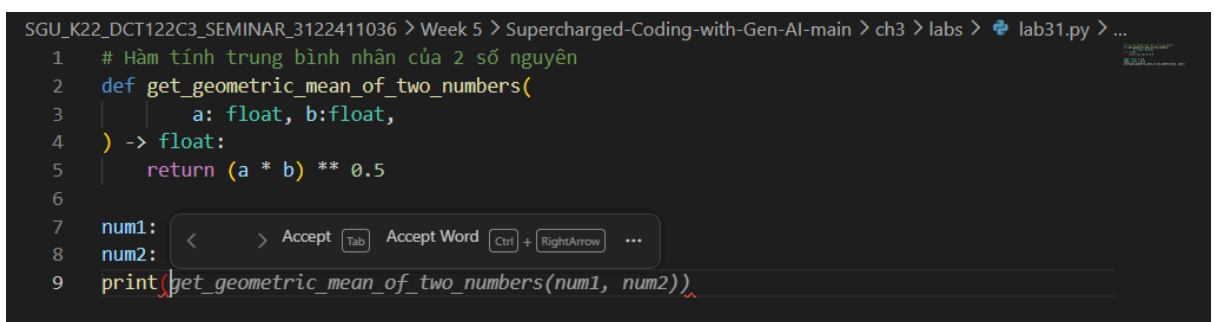
2.2.2 Lab 3.1 – Basic Chat and Code Completion

Mục tiêu của lab này là sử dụng tính năng Chat để hiểu lý thuyết, sau đó dùng định dạng hàm làm mẫu (cue) để Copilot tự động hoàn thiện mã tính trung bình nhân của 2 số.

Execution Flow



```
MINAR_3122411036 > Week 5 > Supercharged-Coding-with-Gen-AI-main > ch3 > labs > lab31.py > get_geometric_mean_of_two_numbers
1 # Hàm tính trung bình nhân của 2 số nguyên
2 def get_geometric_mean_of_two_numbers(
3     a: float, b:float,
4 ) -> float:
5     return (a * b) ** 0.5
```



```
SGU_K22_DCT122C3_SEMINAR_3122411036 > Week 5 > Supercharged-Coding-with-Gen-AI-main > ch3 > labs > lab31.py > ...
1 # Hàm tính trung bình nhân của 2 số nguyên
2 def get_geometric_mean_of_two_numbers(
3     a: float, b:float,
4 ) -> float:
5     return (a * b) ** 0.5
6
7 num1:
8 num2:
9 print(get_geometric_mean_of_two_numbers(num1, num2))
```

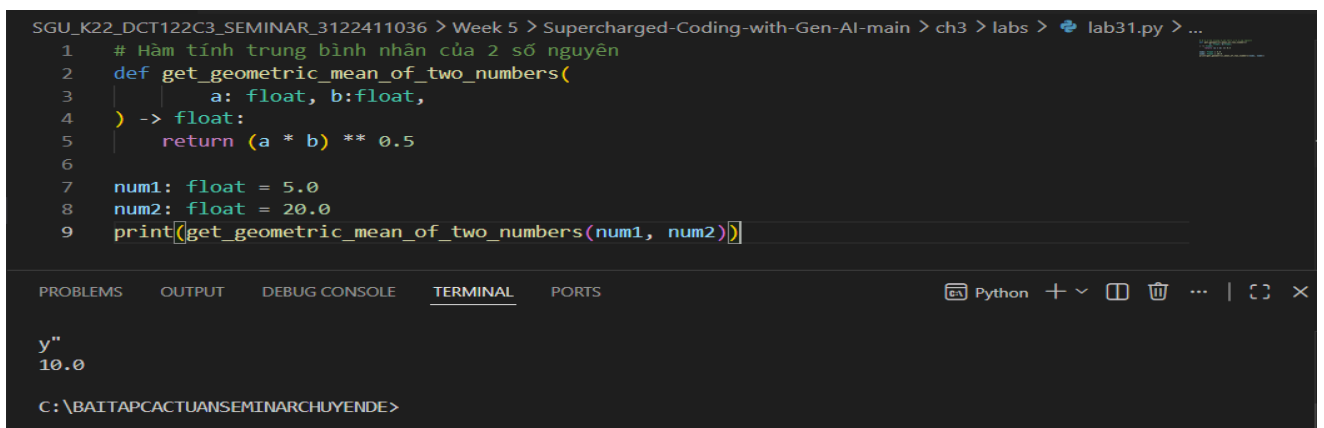
Code Implementation

```
def get_geometric_mean_of_two_numbers(  
    a: float,  
    b: float,  
) -> float:  
    return pow(a * b, 1/2)  
  
num1: float = 5.0  
num2: float = 20.0  
print(get_geometric_mean_of_two_numbers(num1, num2))
```

Code trên thực hiện:

- Định nghĩa hàm với các tham số kiểu float làm gợi ý cho AI.
- Chấp nhận gợi ý mã nội tuyến tính căn bậc 2 của tích.
- Sử dụng lệnh print làm tín hiệu để AI tự động gọi hàm với dữ liệu mẫu.

Program Output



The screenshot shows a code editor with a dark theme. The top part displays the Python code for calculating the geometric mean of two numbers. The bottom part shows the output of the program, which is '10.0'. The editor interface includes a terminal window at the bottom with the command prompt 'C:\BAITAPCACTUANSEMINARCHUYENDE>'.

```
SGU_K22_DCT122C3_SEMINAR_3122411036 > Week 5 > Supercharged-Coding-with-Gen-AI-main > ch3 > labs > lab31.py > ...  
1 # Hàm tính trung bình nhân của 2 số nguyên  
2 def get_geometric_mean_of_two_numbers(  
3     a: float, b:float,  
4 ) -> float:  
5     return (a * b) ** 0.5  
6  
7 num1: float = 5.0  
8 num2: float = 20.0  
9 print(get_geometric_mean_of_two_numbers(num1, num2))  
  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
y"  
10.0  
C:\BAITAPCACTUANSEMINARCHUYENDE>
```

2.2.3 Keyboard Shortcuts and Completion Pane

Để tăng tốc độ làm việc, developer cần sử dụng các phím tắt logic để duyệt các kết quả trả về từ Copilot:

- Chấp nhận toàn bộ: Tab.
- Chấp nhận từng dòng: Trỏ chuột chọn Accept Line.
- Chấp nhận từng từ: Ctrl + Mũi tên phải.
- Chuyển đổi gợi ý: Alt +] hoặc Alt + [.

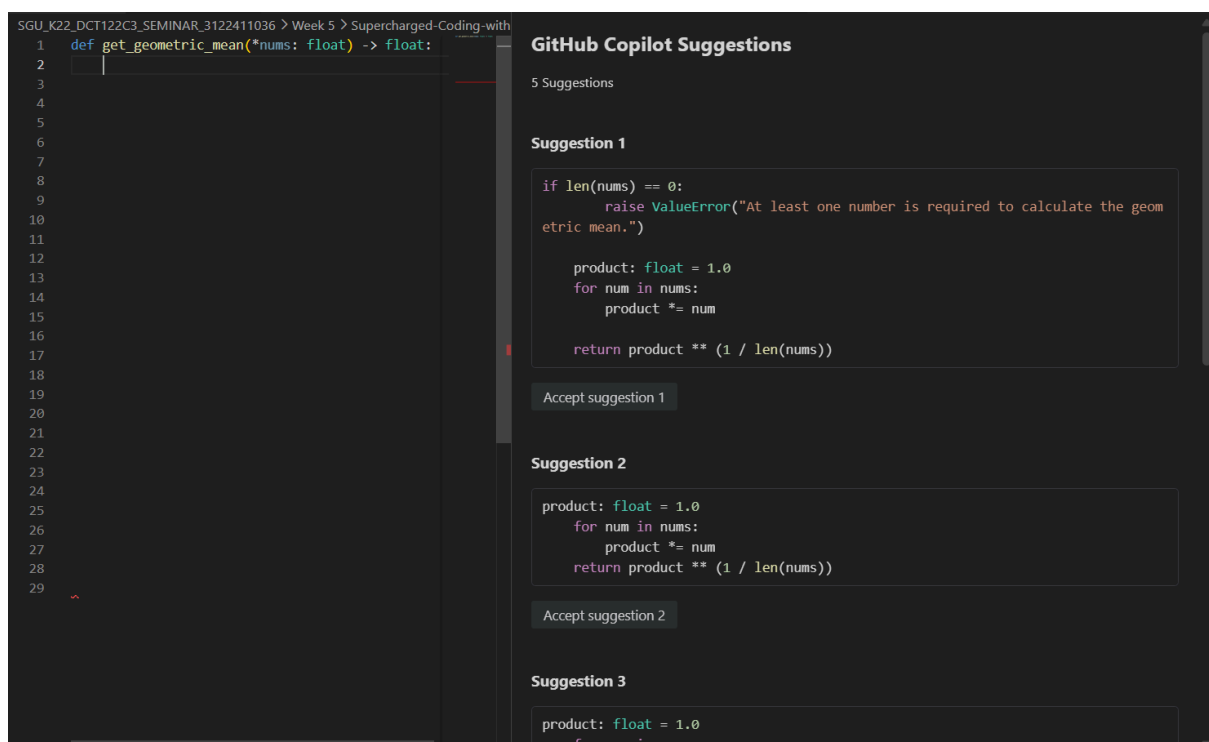
Đặc biệt, sử dụng Ctrl + Enter sẽ mở ra Completion Pane hiển thị lên tới 10 luồng logic giải quyết vấn đề khác nhau.

2.2.4 Lab 3.2 – Advanced Completion for Sequences

Lab này tập trung vào việc tính trung bình nhân cho một chuỗi nhiều số (*nums) và thực hành lựa chọn logic mã tốt nhất từ Completion Pane.

Parameter Flow

```
SGU_K22_DCT122C3_SEMINAR_3122411036 > Week 5 > Supercharged-Coding-with-Gen-AI-main > ch3 > labs > lab32.py > ...
1 def get_geometric_mean(*nums: float) -> float:
2     product: float = 1.0
3     for num in nums:
4         product *= num
5     return product ** (1 / len(nums))
```



2.2.5 Copilot Edits – Inline modifications

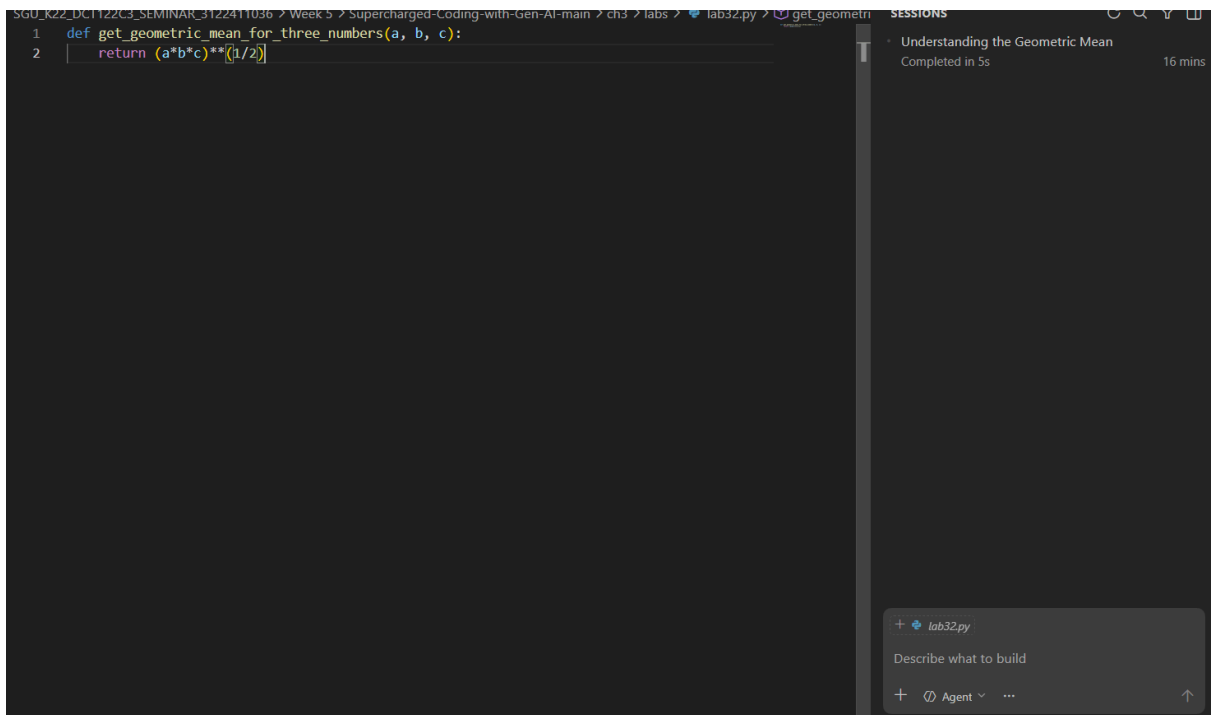
Tính năng này cho phép chỉnh sửa code thông qua thẻ tham chiếu (annotation) `#selection`.

- Developer bôi đen hàm và sử dụng lệnh: `#selection` add input validation for float inputs.
- Copilot hiển thị đề xuất chỉnh sửa dưới dạng so sánh phiên bản (version control format) để developer duyệt và bấm Accept.

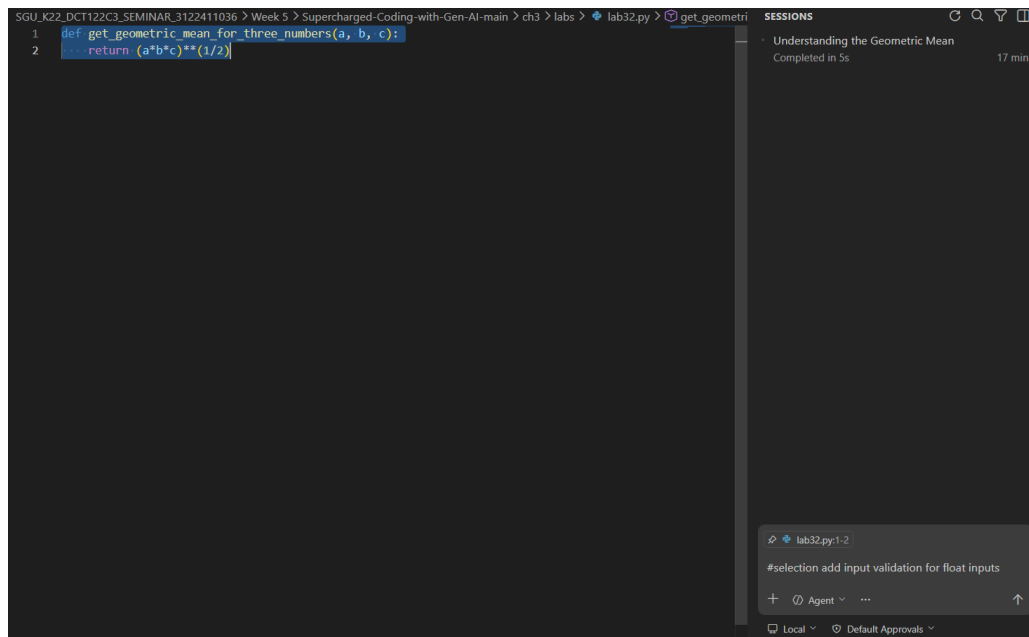
Chuẩn bị đoạn code:

```
def get_geometric_mean_for_three_numbers(a, b, c):  
    return (a*b*c)**(1/2)
```

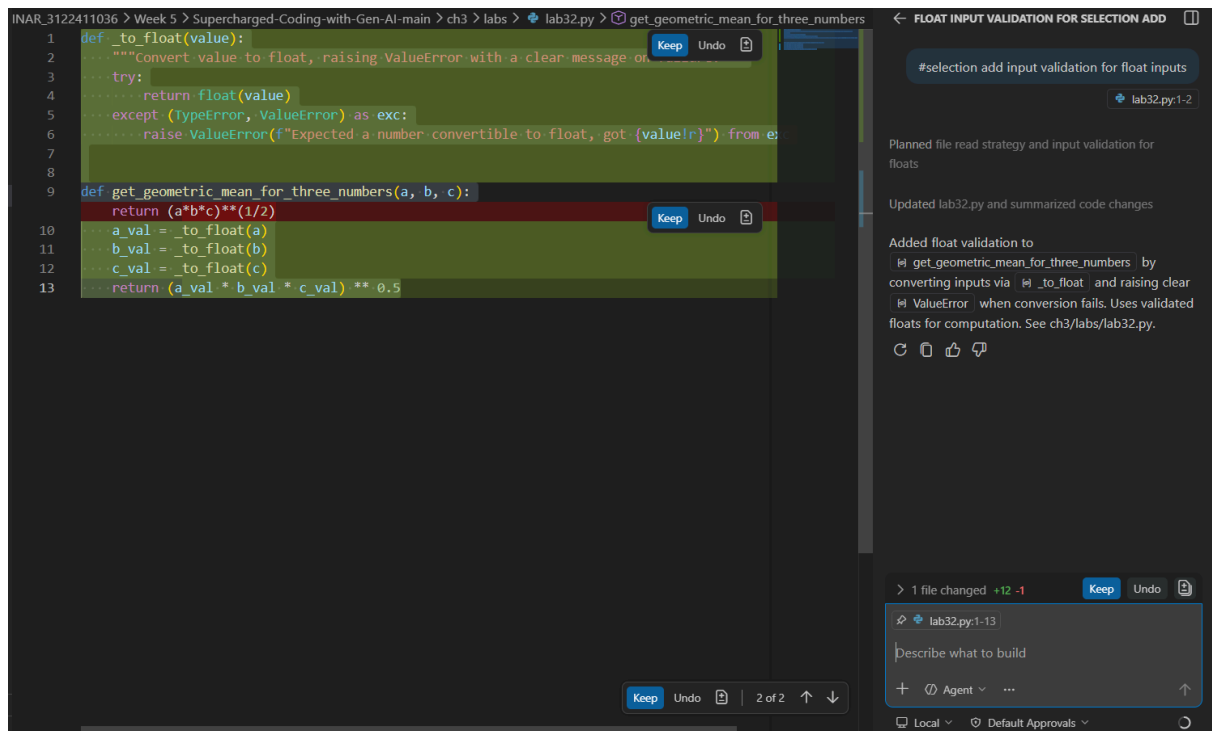
- **Mở Copilot Edits:** Mở cửa sổ Chat của Copilot bên thanh công cụ (Sidebar) của VS Code, sau đó bấm chọn tab **Edit**.



- **Sử dụng thẻ tham chiếu (#selection):** Bôi đen hàm vừa viết trong file code. Trong ô nhập lệnh của tab Edit, bạn gõ chính xác câu lệnh (prompt) sau:
#selection add input validation for float inputs.. Cú pháp này yêu cầu AI tự động sinh thêm các dòng code kiểm tra lỗi để đảm bảo tham số truyền vào bắt buộc phải là số thực (float).



- **Duyệt và chấp nhận code:** Copilot sẽ không ghi đè trực tiếp ngay lập tức mà hiển thị đề xuất dưới dạng so sánh phiên bản (version control format) giống hệt như khi bạn xem diff trong Git. Bạn có thể xem lại từng đoạn thay đổi (phần màu xanh lá là phần AI thêm vào) và nhấn nút **Accept** để áp dụng.



2.2.6 Lab 3.3 – Code Analysis with Slash Commands Mục tiêu

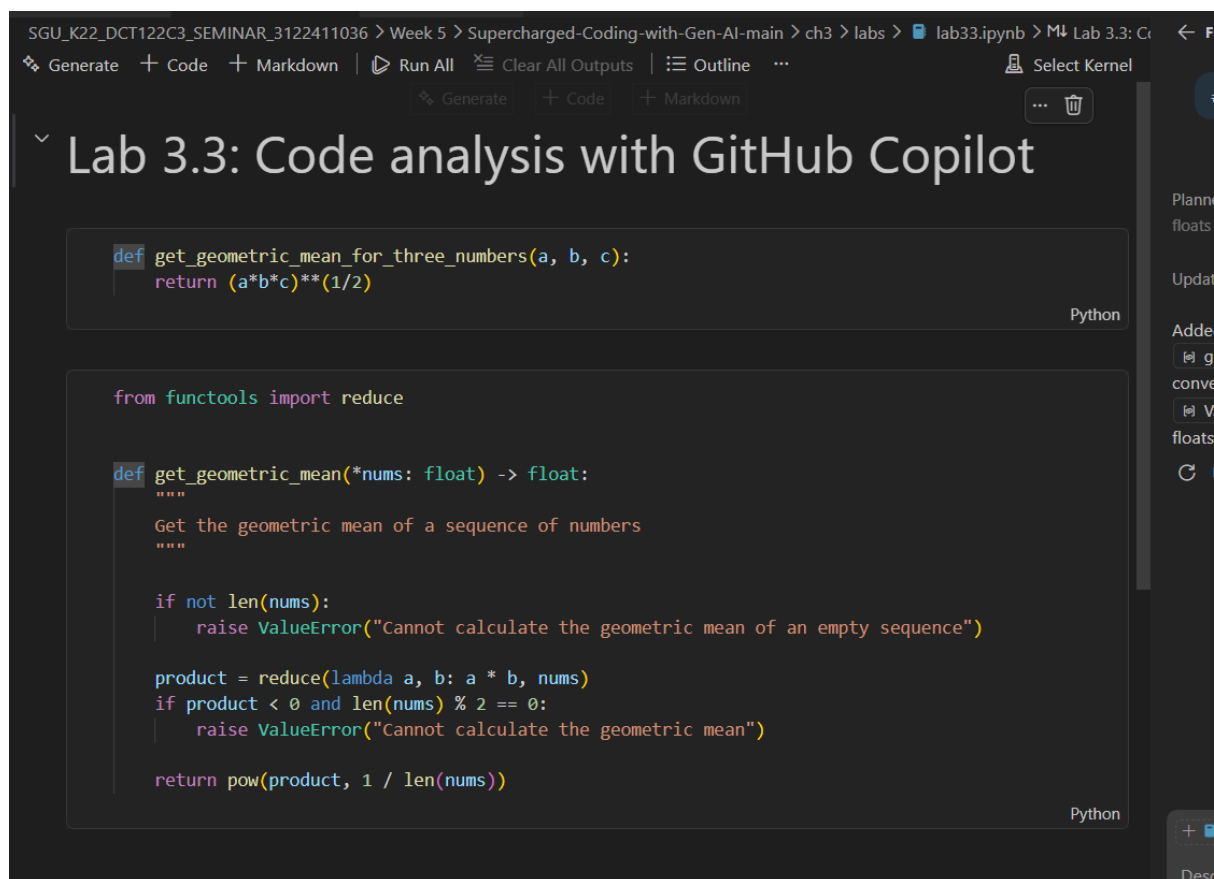
Lab này minh họa cách sử dụng giao diện Chat nội tuyến (Inline Chat) trên VS Code để phân tích và gỡ lỗi code phức tạp. AI được sử dụng như một nhà phân tích mã, có khả năng:

- Phát hiện lỗi logic toán học.
- Đề xuất bản vá lỗi (/fix).
- Giải thích mã nguồn thành ngôn ngữ tự nhiên (/explain).

Workflow Logic Ở mức logical level, chương trình thực hiện các bước sau:

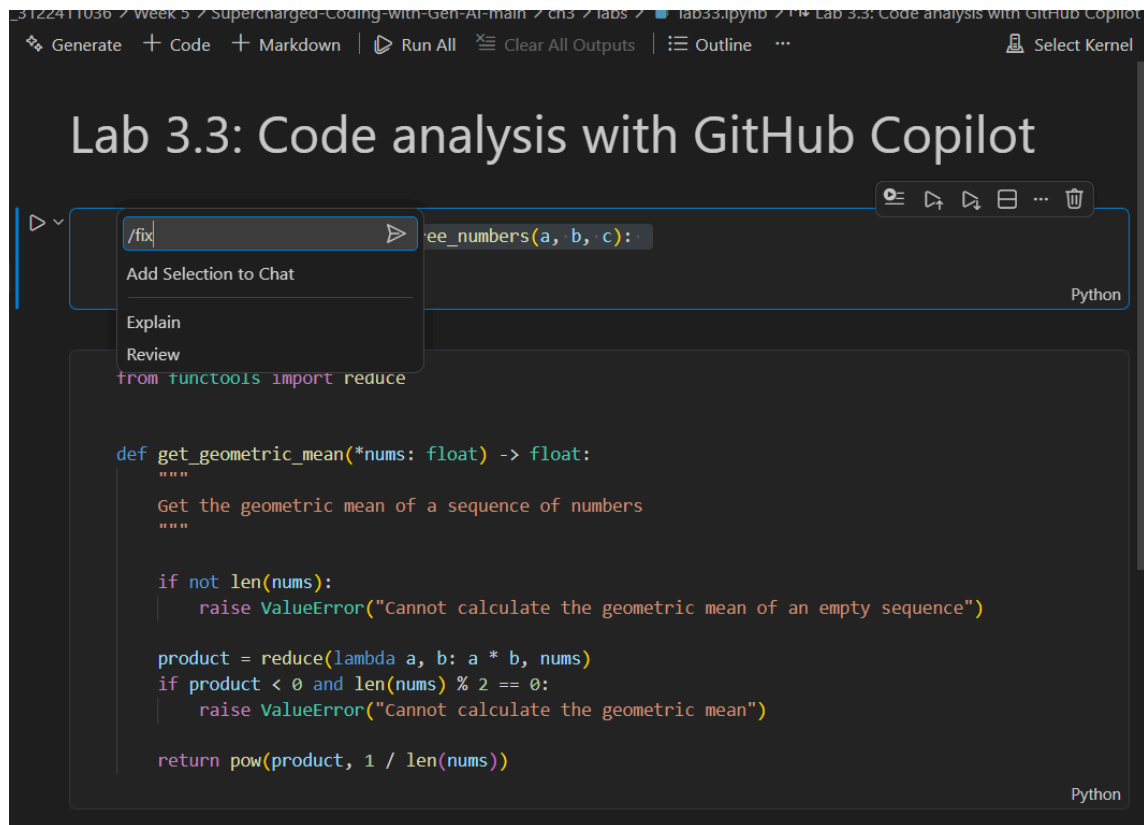
- Người dùng bôi đen đoạn mã bị lỗi.
- Nhấn Ctrl + I gọi Inline Chat và gõ lệnh /fix.
- Copilot phân tích và đề xuất sửa toán tử (1/2) thành (1/3) hoặc mean (a,b,c) để tính căn bậc 3 cho 3 tham số.

Execution Flow



Ta sẽ dùng lệnh /fix để sửa lỗi này bằng các thao tác sau đây:

- **Thao tác:** Bạn bôi đen toàn bộ hàm đầu tiên.
- Mở khung chat nội tuyến (Inline Chat) trên VS Code bằng tổ hợp phím Ctrl + I (hoặc Cmd + I trên Mac).
- Gõ lệnh /fix vào khung chat và nhấn Enter.



The screenshot shows the VS Code interface with an inline chat window open. The chat window has a search bar with the text "/fix" and a list of actions: "Add Selection to Chat", "Explain", and "Review". The code editor displays a Python function definition for "get_geometric_mean".

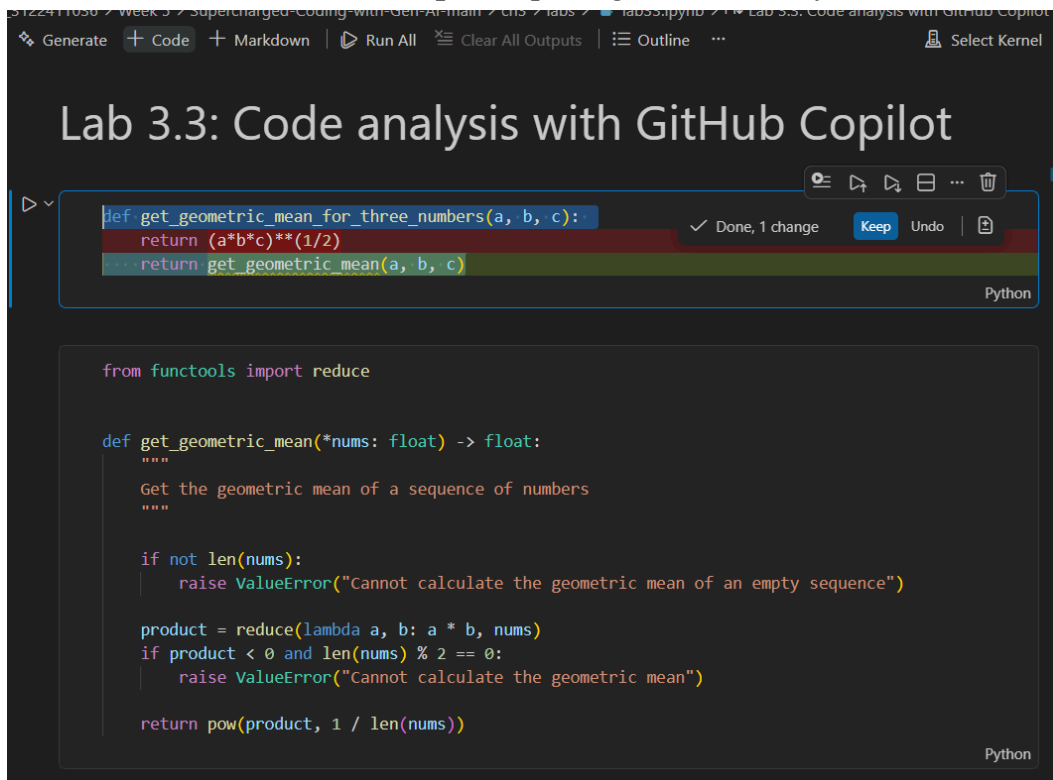
```
from functools import reduce

def get_geometric_mean(*nums: float) -> float:
    """
    Get the geometric mean of a sequence of numbers
    """
    if not len(nums):
        raise ValueError("Cannot calculate the geometric mean of an empty sequence")

    product = reduce(lambda a, b: a * b, nums)
    if product < 0 and len(nums) % 2 == 0:
        raise ValueError("Cannot calculate the geometric mean")

    return pow(product, 1 / len(nums))
```

- **Kết quả:** Copilot sẽ tự động phân tích lỗi toán học và đề xuất mã sửa chữa thay thế. Bạn chỉ cần nhấn nút **Accept** để áp dụng bản vá lỗi này.



The screenshot shows a code editor interface with a dark theme. At the top, there's a title bar with various icons and text: "12:24 PM 10/30 > week 3 > Supercharged Coding with Gen AI - main > CHS > labs > lab33.python > Lab 3.3: Code analysis with GitHub Copilot". Below the title bar, there's a toolbar with buttons: "Generate", "+ Code", "+ Markdown", "Run All", "Clear All Outputs", "Outline", and "Select Kernel". The main area of the editor is titled "Lab 3.3: Code analysis with GitHub Copilot". It shows a Python function definition:

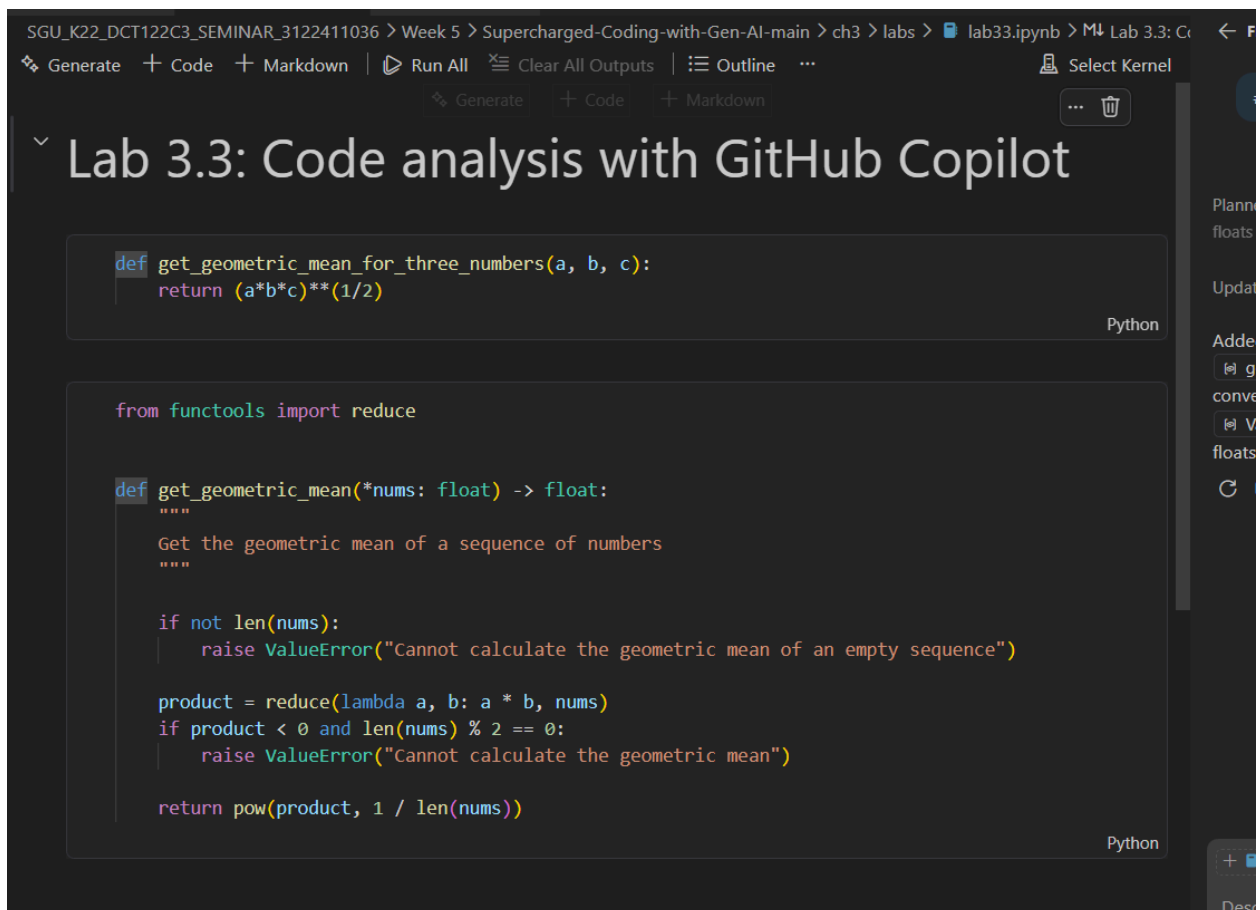
```
def get_geometric_mean_for_three_numbers(a, b, c):  
    return (a*b*c)**(1/2)  
    return get_geometric_mean(a, b, c)
```

 A tooltip above the second line indicates a change: "Done, 1 change" with a "Keep" button. Below this, the full function definition is shown:

```
from functools import reduce  
  
def get_geometric_mean(*nums: float) -> float:  
    """  
    Get the geometric mean of a sequence of numbers  
    """  
  
    if not len(nums):  
        raise ValueError("Cannot calculate the geometric mean of an empty sequence")  
  
    product = reduce(lambda a, b: a * b, nums)  
    if product < 0 and len(nums) % 2 == 0:  
        raise ValueError("Cannot calculate the geometric mean")  
  
    return pow(product, 1 / len(nums))
```

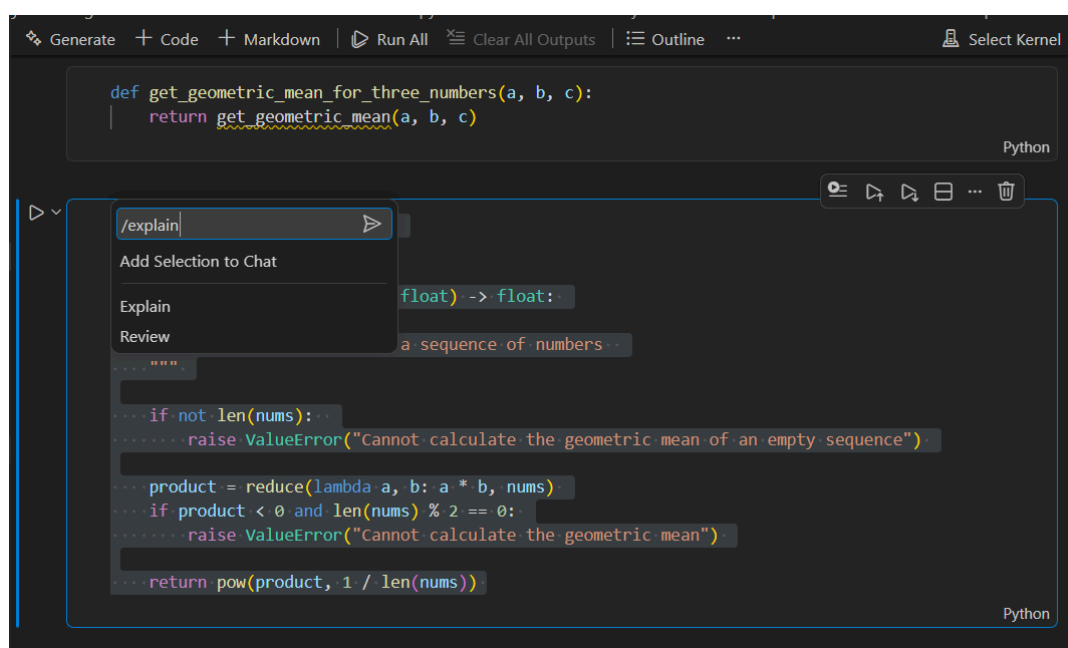
Dùng lệnh /explain để dịch code phức tạp thành ngôn ngữ tự nhiên

Hàm thứ hai (get_geometric_mean) có logic phức tạp hơn khi dùng hàm reduce và các điều kiện kiểm tra bất lỗi (raise ValueError).



Ta sẽ dùng lệnh /explain để dịch code cho dễ hiểu hơn

- **Thao tác:** Bôi đen toàn bộ hàm thứ hai.
- Nhấn Ctrl + I để gọi Inline Chat.
- Gõ lệnh /explain và nhấn Enter.



- **Kết quả:** Copilot sẽ xuất ra một đoạn văn bản giải thích cặn kẽ cách hàm hoạt động: từ việc kiểm tra mảng rỗng ban đầu, cách dùng reduce để nhân dồn các số, cho đến lý do bắt lỗi nếu tích là số âm mà độ dài mảng lại là số chẵn.

```
def get_geometric_mean_for_three_numbers(a, b, c):
    return (a*b*c)**(1/3)
```

```
from functools import reduce

def get_geometric_mean(*nums: float) -> float:
    """
    Get the geometric mean of a sequence of numbers
    """
    # Reject empty input because geometric mean needs at least one value.
    if not len(nums):
        raise ValueError("Cannot calculate the geometric mean of an empty sequence")

    # Multiply all numbers first, then take the n-th root of the product.
    product = reduce(lambda a, b: a * b, nums)

    # For an even count, a negative product would require a complex result.
    if product < 0 and len(nums) % 2 == 0:
        raise ValueError("Cannot calculate the geometric mean")

    # Compute geometric mean: (x1 * x2 * ... * xn)^(1/n).
    return pow(product, 1 / len(nums))
```

2.3 Physical Level

2.3.1 Installing Copilot Extension

Cài đặt phần mở rộng thông qua VS Code Marketplace:

- Tìm kiếm và cài đặt GitHub Copilot.
- Đăng nhập tài khoản GitHub để xác thực giấy phép.

2.3.2 Example Copilot Architecture in VS Code

Sử dụng trực tiếp các tổ hợp phím mức vật lý để điều khiển AI:

- Ctrl + I để gọi khung chat nội tuyến.
- Ctrl + Enter để mở panel gợi ý đa luồng.

3. Summary

Chương này đã trình bày cách sử dụng GitHub Copilot từ góc độ phương pháp luận 3 tầng đến triển khai thực tế trên VS Code. Các nội dung chính bao gồm:

- Giới thiệu AI Pair Programming và kỹ thuật trích xuất ngữ cảnh.
- Thực hiện sinh mã tự động thông qua Function signature.
- Làm chủ các phím tắt để duyệt và lựa chọn logic mã tối ưu trong màn hình Completion Pane.
- Phân tích, giải nghĩa và gỡ lỗi mã nguồn phức tạp bằng các lệnh /fix và /explain. Thông qua các bài lab, người học có thể hiểu rõ cách áp dụng phương pháp Vibe Coding vào thực tế để tăng tốc chu kỳ phát triển phần mềm Python Backend.

CHƯƠNG 4: Best Practices for Prompting with ChatGPT

1. Introduction

Trong lập trình với Generative AI (GenAI), việc đặt câu hỏi cho mô hình ngôn ngữ như ChatGPT đóng vai trò quan trọng trong việc tạo ra kết quả chính xác.

Theo sách *Supercharged Coding with GenAI*, chất lượng đầu ra của mô hình phụ thuộc nhiều vào cách thiết kế prompt.

Prompt Engineering là quá trình thiết kế prompt sao cho mô hình hiểu chính xác yêu cầu của người dùng và tạo ra kết quả mong muốn. Việc xây dựng prompt hiệu quả giúp tăng độ chính xác của kết quả và giảm thời gian chỉnh sửa.

Để đạt hiệu quả cao, chương này đề xuất **framework Five S's** nhằm chuẩn hóa cách viết prompt. Framework này gồm 5 nguyên tắc chính:

Five S Framework for Prompt Engineering

S tructured format

Prompt cần có cấu trúc rõ ràng để mô hình hiểu nhiệm vụ.
Cấu trúc thường gồm:

- Task – mô tả nhiệm vụ cần thực hiện
- Context – cung cấp bối cảnh
- Instruction – yêu cầu cụ thể

S urrounding context

Cung cấp thông tin ngữ cảnh liên quan để mô hình hiểu vấn đề.
Ngữ cảnh có thể bao gồm:

- Môi trường hệ thống
- Phiên bản phần mềm
- Dữ liệu liên quan

S ingle-task focus

Prompt nên tập trung vào một nhiệm vụ duy nhất.
Điều này giúp:

- Giảm sự mơ hồ
- Tăng độ chính xác của kết quả

S pecific instructions

Prompt cần mô tả rõ ràng và chi tiết yêu cầu của nhiệm vụ, bao gồm các bước hoặc dữ liệu liên quan, thay vì đưa ra một yêu cầu chung chung.

S hort prompt

Prompt nên ngắn gọn và loại bỏ thông tin không cần thiết.
Prompt ngắn giúp:

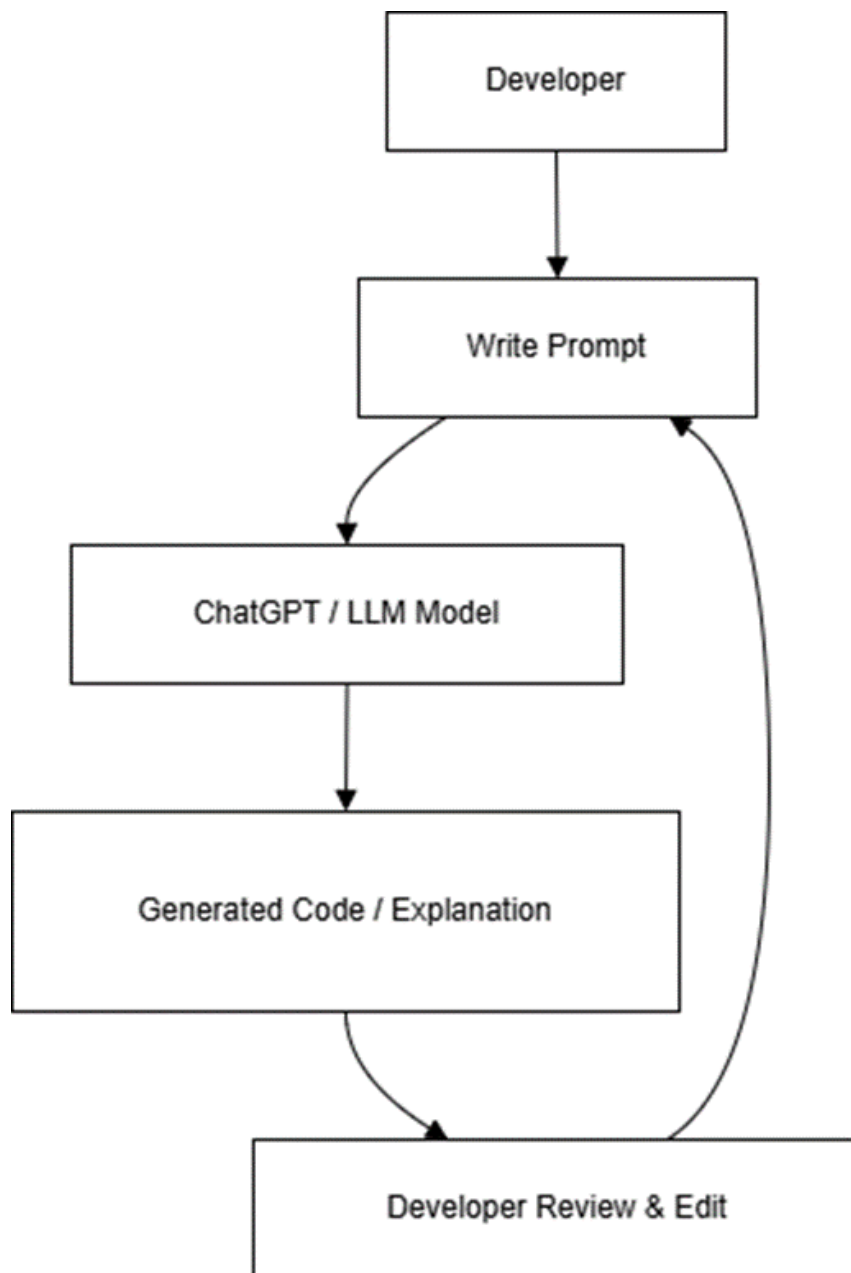
- giảm nhiễu thông tin
- tăng độ chính xác của phản hồi

2. Methodology

2.1. Conceptual Level

Ở mức ý tưởng, hệ thống được mô tả dưới góc nhìn tổng quát về cách GenAI hỗ trợ lập trình viên.

Trong mô hình này, lập trình viên gửi yêu cầu dưới dạng prompt đến hệ thống GenAI. Mô hình sẽ phân tích ngữ cảnh của prompt và sinh ra kết quả tương ứng như mã nguồn, giải thích code hoặc gợi ý sửa lỗi.



Sơ đồ này thể hiện cách lập trình viên tương tác với hệ thống AI để nhận hỗ trợ trong quá trình phát triển phần mềm.

2.2. Logical Level

Ở mức luận lý, ta mô tả cách prompt được cấu trúc và xử lý.

Một prompt thường gồm các thành phần:

- System Prompt - xác định vai trò của mô hình
- User Prompt - mô tả nhiệm vụ
- Input Data - dữ liệu hoặc đoạn code cần xử lý

Ví dụ:

“System: You are a senior Python developer
User: Fix the bug in the following code”

Một số kỹ thuật prompt khác

- **Zero-shot Prompting**
 - Zero-shot prompting là kỹ thuật yêu cầu mô hình AI thực hiện nhiệm vụ mà không cung cấp ví dụ trước đó. Mô hình sẽ dựa vào kiến thức đã được huấn luyện để đưa ra câu trả lời. Kỹ thuật này phù hợp với ChatGPT vì mô hình có khả năng xử lý nhiều loại câu hỏi khác nhau trong môi trường hội thoại. Người dùng thường chỉ cần mô tả nhiệm vụ là mô hình có thể đưa ra phản hồi.
- **Few-shot Prompting**
 - Few-shot prompting cung cấp một số ví dụ mẫu trước khi yêu cầu mô hình thực hiện nhiệm vụ. Những ví dụ này giúp mô hình hiểu định dạng và cách xử lý mong muốn. ChatGPT phản hồi tốt khi được cung cấp ví dụ minh họa. Điều này đặc biệt hữu ích trong các nhiệm vụ như phân loại dữ liệu hoặc chuyển đổi định dạng.
- **Chain-of-Thought Prompting**
 - Chain-of-Thought prompting yêu cầu mô hình **trình bày quá trình suy luận từng bước** trước khi đưa ra kết quả cuối cùng. ChatGPT thường được dùng để giải thích logic hoặc debugging code. Chain-of-Thought giúp mô hình suy luận rõ ràng hơn trong các nhiệm vụ phức tạp.

2.3. Physical Level

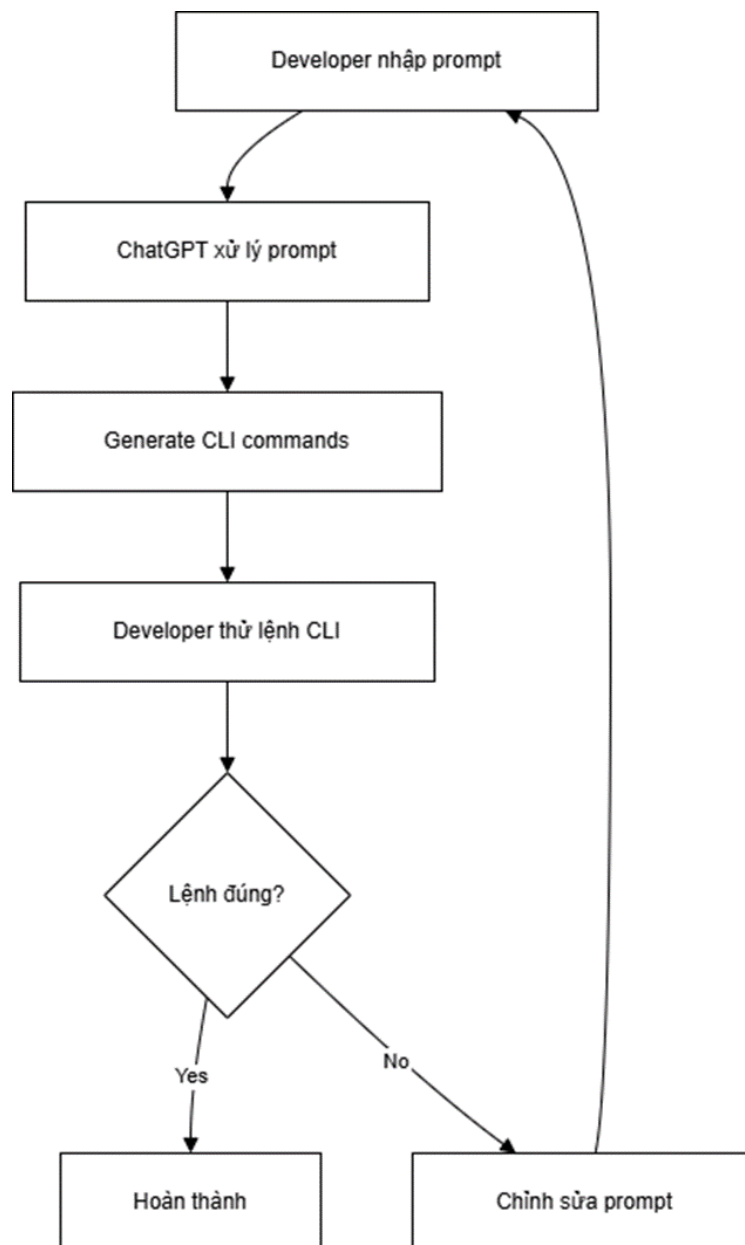
Trong phần này, các bài **lab thực nghiệm** được thực hiện để kiểm tra khả năng của GenAI trong các nhiệm vụ lập trình.

Lab 4.1 – Converting PyCharm GUI steps to CLI

Mục tiêu

Sử dụng Chat GPT để chuyển đổi các bước thao tác bằng giao diện đồ họa (GUI) sang lệnh dòng lệnh (CLI).

Luồng hoạt động



Prompt

Prompt bình thường

How can I commit my files to GitHub from PyCharm terminal?

Kết quả

How can I commit my files to GitHub from PyCharm terminal?

To commit files to GitHub using the PyCharm terminal, you are essentially using standard Git CLI commands inside the integrated terminal. The workflow normally consists of 4 main steps:

1. Check repository status
2. Add files to staging
3. Commit changes
4. Push to GitHub

Below is the exact command sequence.

1. Open the PyCharm Terminal

In PyCharm:

```
sql
```

Sao chép mã

View → Tool Windows → Terminal

or press:

```
nginx
```

Sao chép mã

Alt + F12

This opens a terminal inside your project directory.

2. Check Git Status

Run:

```
bash
```

Sao chép mã

```
git status
```

This shows:

- modified files
- new files
- files already staged

Example output:

```
makefile
```

Sao chép mã

```
modified: lab42.py
untracked: utils.py
```

3. Add Files to Staging

Add all changes:

```
bash
```

 Sao chép mã

```
git add .
```

Or add a specific file:

```
bash
```

 Sao chép mã

```
git add lab42.py
```

Explanation:

Command	Meaning
<code>git add .</code>	stage all changed files
<code>git add file.py</code>	stage a single file

4. Commit the Changes

Create a commit with a message:

bash

 Sao chép mã

```
git commit -m "Add cost estimation function and comments"
```

Best practice:

Use short but meaningful commit messages.

Examples:

sql

 Sao chép mã

```
git commit -m "Fix token estimation bug"
git commit -m "Add prompt generation function"
git commit -m "Refactor get_estimated_user_costs"
```

5. Push to GitHub

Send commits to the remote repository:

bash

 Sao chép mã

```
git push origin main
```

or if your branch is master:

bash

 Sao chép mã

```
git push origin master
```

Complete Workflow Example

bash

 Sao chép mã

```
git status
git add .
git commit -m "Implement prompt generator and cost estimator"
git push origin main
```

Prompt có kỹ thuật

CONTEXT: You will be provided with PyCharm GUI steps enclosed with {{{STEPS }}} to execute a process enclosed with {{{ PROCESS }}}.

TASK: Convert the steps to CLI commands.

PROCESS:

{{{ Commit and push files to a remote Git branch }}}}

STEPS: {{{

1. Review changed files
2. Stage the desired files
3. Add a commit message
4. Commit the files
5. Validate branch name
6. Push the changes to the remote branch

}}}

CLI COMMANDS:

Kết quả

3. Add a commit message

In CLI this is done together with the commit command.

4. Commit the files

bash

 Sao chép mã

```
git commit -m "Your commit message"
```

Example:

bash

 Sao chép mã

```
git commit -m "Add comments explaining each step of the function"
```

5. Validate branch name

Check which branch you are currently on.

bash

 Sao chép mã

```
git branch
```

or

bash

 Sao chép mã

```
git branch --show-current
```

6. Push the changes to the remote branch

bash

Sao chép mã

```
git push origin <branch-name>
```

Example:

bash

Sao chép mã

```
git push origin main
```

Complete CLI sequence (typical workflow)

bash

Sao chép mã

```
git status
git add .
git commit -m "Your commit message"
git branch --show-current
git push origin <branch-name>
```

Phân tích

Khi sử dụng prompt đơn giản:

“How can I commit my files to GitHub from PyCharm terminal?”

ChatGPT đưa ra hướng dẫn tổng quát nhưng không đảm bảo bám sát đúng quy trình thao tác đã mô tả trong GUI.

Khi sử dụng prompt có kỹ thuật giúp mô hình hiểu rõ:

- Quy trình cần chuyển đổi
- Từng bước cụ thể
- Định dạng đầu ra mong muốn

So với prompt ban đầu, prompt có cấu trúc:

- Tạo kết quả chính xác hơn
- Giữ đúng thứ tự thao tác
- Giảm việc giải thích dư thừa

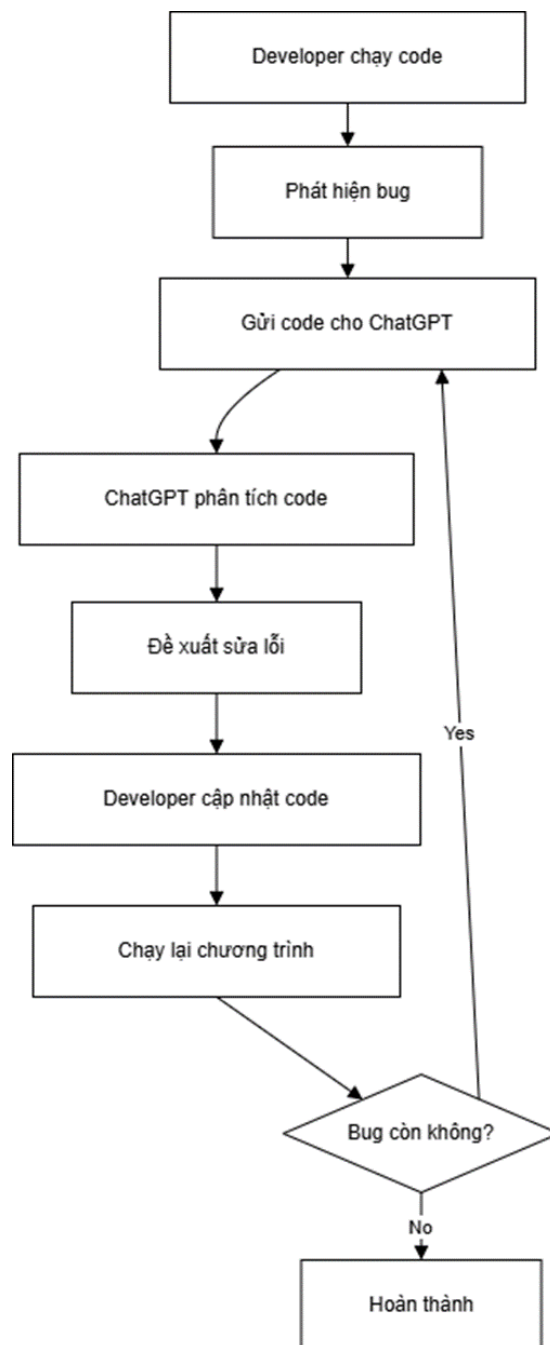
Lab này cho thấy **structured prompting** giúp mô hình hiểu rõ nhiệm vụ hơn, từ đó tạo ra kết quả phù hợp với quy trình kỹ thuật.

Lab 4.2 - Code debugging

Mục tiêu

Sử dụng GenAI để phát hiện và sửa lỗi cú pháp trong code Python.

Luồng hoạt động



Code

```
import inspect

from openai import OpenAI
from openai.types.chat import ChatCompletion

SURROUND = """
You are a senior Python developer and code reviewer.

Your job is to analyze Python functions and help developers improve them.
You should identify logical bugs, incorrect assumptions, inefficiencies,
and suggest better implementations following Python best practices.

When reviewing code:
- explain what is wrong
- explain why it is wrong
- propose a corrected or improved version
- keep explanations concise and technical
"""

SINGLE_TASK = """
Analyze the given Python function.

Tasks:
1. Identify any logical bugs or incorrect assumptions.
2. Point out inefficient or non-pythonic code patterns.
3. Suggest improvements for readability and maintainability.
4. Provide a corrected or optimized version of the function.

Return the explanation first, then the improved code.
"""

def get_user_prompt(func: callable, change: str) -> str:
    source = inspect.getsource(func)
    return f"""
{SURROUND}

TASK:
{SINGLE_TASK}
```

CODE:

```
{source}
```

CHANGE REQUEST:

```
{change}
```

```
"""
```

```
def get_estimated_user_costs(prompts_data):
```

```
    costs = {}
```

```
    for entry in prompts_data:
```

```
        user = entry["user"]
```

```
        prompt = entry["prompt"]
```

```
        estimated_tokens = len(prompt) / 4 # 4 is the average number of tokens per  
character
```

```
        cost = estimated_tokens * 0.0001 # 0.0001 is the cost per token
```

```
    if user not in costs:
```

```
        costs[user] = cost
```

```
    else:
```

```
        costs[user] += cost
```

```
    return costs
```

```
if __name__ == "__main__":
```

```
    client: OpenAI = OpenAI()
```

```
    changes = ["Debug the function and identify any logical errors."]
```

```
    for c in changes:
```

```
        user_prompt = get_user_prompt(get_estimated_user_costs, c)
```

```
    completion: ChatCompletion = client.chat.completions.create(
```

```
        model="gpt-4o-mini",
```

```
        messages=[{"role": "system", "content": SURROUND},
```

```
                    {"role": "user", "content": user_prompt},],
```

```
        temperature=0.2,
```

```
    )
```

```
print(f'Change: {c}')
print(completion.choices[0].message.content)
```

Kết quả

```
Change: Debug the function and identify any logical errors.
### Analysis
```

- Logical Bugs or Incorrect Assumptions:**
 - The calculation of `estimated_tokens` assumes that 4 characters correspond to 1 token. This is an oversimplification and can lead to inaccurate cost estimations, as the average number of characters per token can vary significantly depending on the language and context.
 - The function does not handle cases where `prompts_data` might be empty or not structured as expected (e.g., missing keys).
- Inefficient or Non-Pythonic Code Patterns:**
 - The use of a dictionary to accumulate costs is fine, but the check for whether a user exists in the dictionary can be simplified using the `defaultdict` from the `collections` module, which would eliminate the need for the `if` statement.
- Improvements for Readability and Maintainability:**
 - The magic numbers (4 for tokens per character and 0.0001 for cost per token) should be defined as constants at the beginning of the function for better readability.
 - Adding type hints for the function parameters and return type can improve clarity.
 - Implementing error handling for unexpected data structures would make the function more robust.

```
### Improved Code
```

```
```python
from collections import defaultdict
from typing import List, Dict

TOKEN_PER_CHAR = 4
COST_PER_TOKEN = 0.0001

def get_estimated_user_costs(prompts_data: List[Dict[str, str]]) -> Dict[str, float]:
 costs = defaultdict(float)

 for entry in prompts_data:
 user = entry.get("user")
 prompt = entry.get("prompt")

 if user is None or prompt is None:
 continue # Skip entries with missing user or prompt

 estimated_tokens = len(prompt) / TOKEN_PER_CHAR
 cost = estimated_tokens * COST_PER_TOKEN
 costs[user] += cost

 return dict(costs)
```
```

```
### Key Changes Made:
```

- Used `defaultdict` to simplify cost accumulation.
- Defined constants for token calculation and cost for clarity.
- Added error handling for missing keys in the input data.
- Included type hints for better documentation and type checking.

Phân tích

Trong lab này, ChatGPT được sử dụng để phân tích và cải thiện hàm: “get_estimated_user_costs”

Hàm này có nhiệm vụ:

- Duyệt danh sách prompt
- Ước lượng số token
- Tính chi phí theo từng user

ChatGPT đã thực hiện ba bước chính:

1. Phát hiện vấn đề

Mô hình chỉ ra rằng việc ước lượng token bằng công thức:

```
estimated_tokens = len(prompt) / 4
```

chỉ là ước lượng thô, không phản ánh chính xác số token thực tế. Ngoài ra, code còn có một số điểm cần cải thiện:

- Thiếu type hints
- Logic cộng chi phí có thể viết Pythonic hơn
- Thiếu comment

2. Đề xuất cải thiện

ChatGPT đề xuất:

- Sử dụng dictionary update tốt hơn
- Cải thiện readability
- Thêm giải thích logic

3. Sinh code cải tiến

Code sau khi sửa trở nên rõ ràng hơn, dễ maintain, dễ đọc

Lab cho thấy ChatGPT có thể:

- Phát hiện lỗi logic
- Cải thiện code style
- Đề xuất implementation tốt hơn

Điều này chứng minh GenAI có thể đóng vai trò code reviewer hỗ trợ lập trình viên.

3. Summary

Chương 4 trình bày các phương pháp hiệu quả để tương tác với ChatGPT trong quá trình hỗ trợ lập trình. Nội dung tập trung vào việc thiết kế prompt sao cho mô hình hiểu chính xác yêu cầu của người dùng và tạo ra kết quả phù hợp. Framework **Five S's** được giới thiệu như một phương pháp chuẩn hóa cách xây dựng prompt, bao gồm việc cung cấp ngữ cảnh, xác định nhiệm vụ cụ thể và đưa ra hướng dẫn rõ ràng cho mô hình.

Thông qua các thí nghiệm thực nghiệm, chương này minh họa cách ChatGPT có thể hỗ trợ lập trình viên trong các nhiệm vụ như chuyển đổi quy trình thao tác từ giao diện đồ họa sang dòng lệnh và hỗ trợ phân tích, phát hiện lỗi trong mã nguồn. Kết quả cho thấy việc sử dụng prompt có cấu trúc và cung cấp ngữ cảnh đầy đủ giúp mô hình tạo ra kết quả chính xác và hữu ích hơn cho quá trình phát triển phần mềm.

CHƯƠNG 5: Best Practices for Prompting with OpenAI API and GitHub Copilot

1. Introduce

Ngoài việc sử dụng ChatGPT trực tiếp, các mô hình GenAI còn có thể được tích hợp vào các ứng dụng và môi trường phát triển phần mềm thông qua API hoặc các plugin trong IDE.

Chương này trình bày cách sử dụng GenAI thông qua:

- OpenAI API
- GitHub Copilot

2. Methodology

2.1. Conceptual Level

Ở mức ý tưởng, Generative AI được xem như một AI pair programmer hỗ trợ lập trình viên trong quá trình phát triển phần mềm. Thay vì hoạt động như một công cụ độc lập, GenAI đóng vai trò là một cộng tác viên kỹ thuật có khả năng phân tích ngữ cảnh của mã nguồn, hiểu yêu cầu của người dùng và đưa ra các gợi ý phù hợp.

Trong mô hình này, lập trình viên tương tác với hệ thống AI thông qua các nền tảng và công cụ khác nhau, chẳng hạn như OpenAI API hoặc các công cụ hỗ trợ lập trình tích hợp trong môi trường phát triển như GitHub Copilot. Hệ thống AI sẽ tiếp nhận thông tin đầu vào từ lập trình viên, bao gồm prompt, đoạn mã nguồn hoặc mô tả nhiệm vụ, sau đó phân tích ngữ cảnh và tạo ra các phản hồi nhằm hỗ trợ quá trình lập trình.

Thông qua sự tương tác này, Generative AI có thể hỗ trợ lập trình viên trong nhiều hoạt động khác nhau, bao gồm:

- sinh mã nguồn hoặc các đoạn code mẫu
- phát hiện và sửa lỗi trong chương trình
- tạo tài liệu cho mã nguồn như comment hoặc docstring
- gợi ý thuật toán hoặc cách triển khai giải pháp

Mô hình ý tưởng này cho thấy Generative AI hoạt động như một trợ lý lập trình thông minh, giúp tăng năng suất làm việc của lập trình viên và hỗ trợ hiệu quả trong nhiều giai đoạn của quy trình phát triển phần mềm.

2.2. Logical Level

Các kỹ thuật prompting thường được sử dụng trong lập trình gồm:

2.2.1. Prompting Techniques for the OpenAI API

- **Retrieval Augmented Generation (RAG)**
 - Retrieval Augmented Generation (RAG) là kỹ thuật kết hợp truy xuất dữ liệu từ nguồn bên ngoài với khả năng sinh văn bản của mô hình AI. RAG đặc biệt phù hợp với OpenAI API vì API thường được tích hợp vào hệ thống phần mềm có cơ sở dữ liệu riêng.
- **Prompt Chaining**
 - Prompt chaining là kỹ thuật chia một nhiệm vụ lớn thành nhiều prompt nhỏ liên tiếp. Trong các ứng dụng dùng OpenAI API, nhiều tác vụ phức tạp cần được xử lý qua nhiều bước. Prompt chaining giúp xây dựng pipeline AI rõ ràng và dễ kiểm soát.

Ví dụ:

Prompt 1 → Extract key information

Prompt 2 → Analyze extracted data

Prompt 3 → Generate final report

Hiệu quả mang lại:

- Cải thiện độ chính xác
- Dễ debug
- Dễ mở rộng hệ thống

2.2.2. Prompting Techniques for GitHub Copilot

- **Zero-shot Prompting (Context Prompting)**
 - Trong Copilot, zero-shot prompting thường được thực hiện thông qua **comment hoặc tên hàm**. Copilot chủ yếu dựa vào **context code trong file**, vì vậy comment ngắn gọn có thể hoạt động như một prompt.

Hiệu quả mang lại: Giúp Copilot hiểu mục tiêu hàm, tạo code nhanh hơn, giảm thời gian viết code lặp lại

- **Directional Stimulus Prompting**

- Directional stimulus prompting là kỹ thuật **đưa ra tín hiệu định hướng để mô hình hiểu mục tiêu code**. Trong Copilot, điều này thường được thực hiện bằng:
 - + Type hint
 - + Comment
 - + Cấu trúc code

Copilot sử dụng ngữ cảnh trong file code để tạo gợi ý. Directional stimulus giúp mô hình hiểu rõ hơn về mục tiêu của đoạn code.

Hiệu quả mang lại: cải thiện chất lượng code suggestion, giảm lỗi logic, tăng tốc độ lập trình

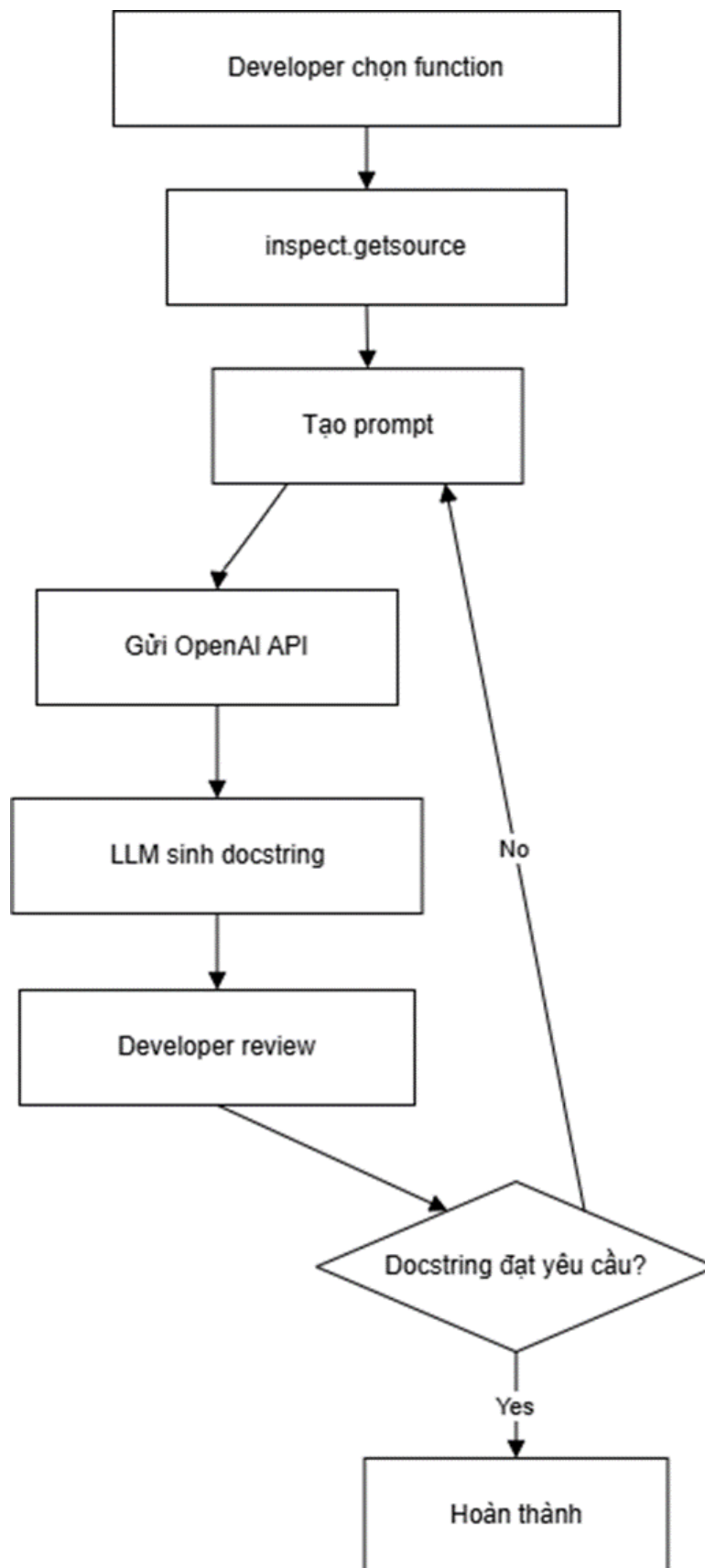
2.3. Physical Level

Lab 5.1 - Generating docstrings with OpenAI API

Mục tiêu

Mục tiêu nghiên cứu khả năng của mô hình Generative AI khi được tích hợp thông qua OpenAI API trong việc tự động tạo tài liệu cho mã nguồn. Cụ thể, thí nghiệm nhằm:

Luồng hoạt động



Code

```
import inspect
```

```
SURROUND = """
```

You are a senior Python developer and technical writer.

You will be provide with a Python function enclosed with {{{ Function }}}.

You write high quality Python documentation and follow best practices for writing clear and structured docstrings.

```
"""
```

```
SINGLE_TASK = """
```

Generate a Python docstring for the provided function.

The docstring should include:

- A short description of the function
- Args section describing parameters
- Returns section describing the output

Use Google style docstrings.

```
"""
```

```
class Singleton(type):
```

```
    _instances = {}
```

```
    def __call__(cls, *args, **kwargs):
```

```

    if cls not in cls._instances:

        cls._instances[cls] = super(Singleton, cls).__call__(*args, **kwargs)

    return cls._instances[cls]

def get_user_prompt(func: callable) -> str:

    return f"""

FUNCTION: {{{{{{ {inspect.getsource(func)} }}}}}}

GOOGLE STYLE DOCSTRING:

"""

if __name__ == "__main__":

    from openai import OpenAI

    from openai.types.chat import ChatCompletion

    client: OpenAI = OpenAI()

    completion: ChatCompletion = client.chat.completions.create(

        model="gpt-4o-mini",

        messages=[{"role": "system", "content": f"{SURROUND}

{SINGLE_TASK}"},

                    {"role": "user", "content": get_user_prompt(Singleton.__call__)},],

    )

```

```
print("Docstring:", completion.choices[0].message.content)
```

Kết quả:

```
Docstring: ```python
def __call__(cls, *args, **kwargs):
    """Create a new instance of the class if one does not already exist.

    This method ensures that only one instance of the class is created
    and returns that instance upon subsequent calls. It utilizes the
    singleton design pattern to manage instance creation.

    Args:
        cls (type): The class being instantiated.
        *args: Variable length argument list to be passed to the class constructor.
        **kwargs: Arbitrary keyword arguments to be passed to the class constructor.

    Returns:
        object: The instance of the class, ensuring that only one instance exists.
    """
    ...
```

Phân tích

Trong thí nghiệm này, OpenAI API được sử dụng để tự động sinh docstring cho hàm Python.

Quy trình thực hiện gồm các bước:

1. Lấy source code của function bằng “inspect.getsource”
2. Tạo prompt chứa function
3. Gửi prompt tới OpenAI API
4. Mô hình sinh docstring theo chuẩn Google style

Kết quả cho thấy mô hình có thể:

- Hiểu chức năng của hàm
- Phân tích tham số
- Mô tả giá trị trả về

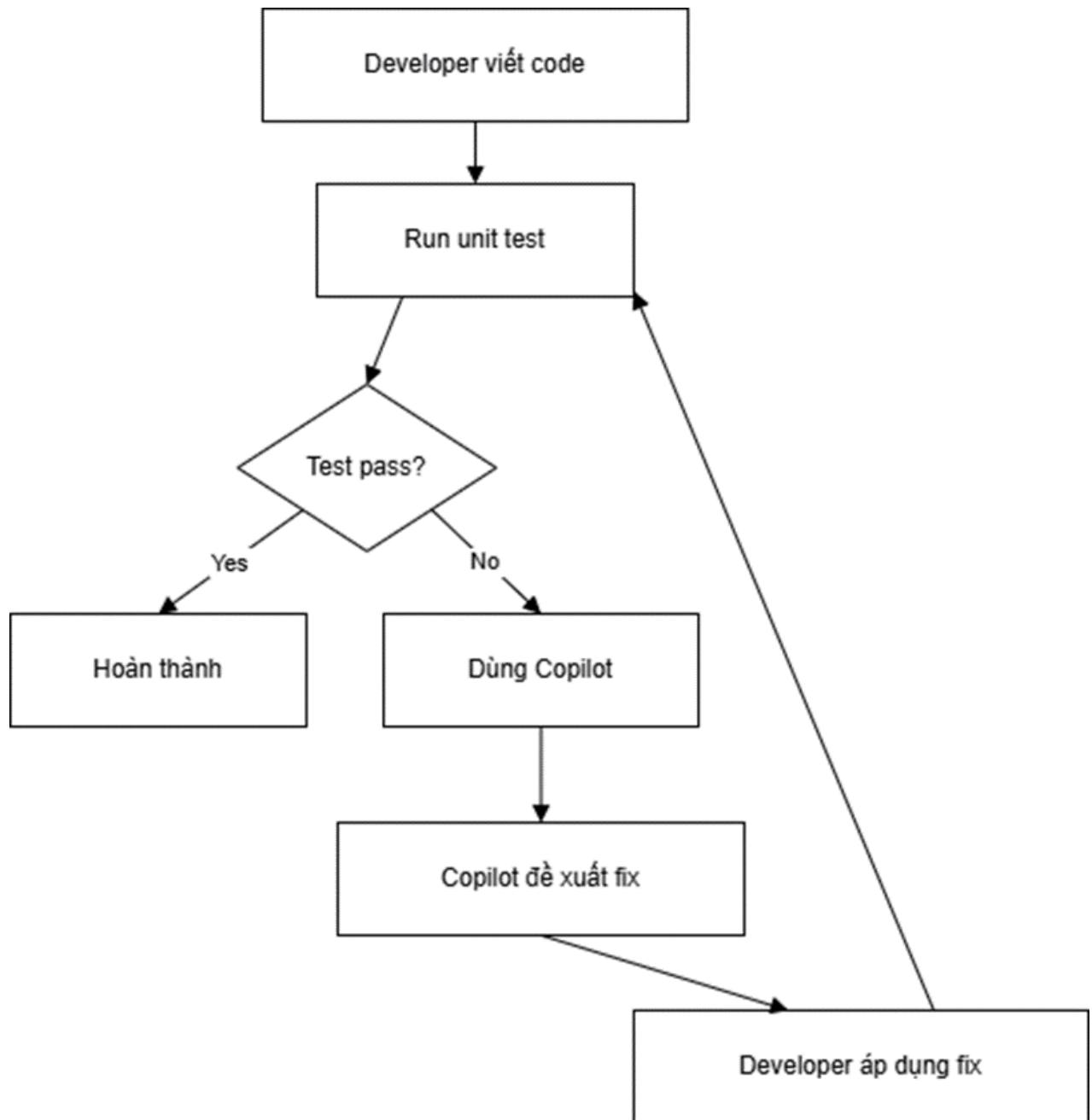
Lab này chứng minh rằng GenAI có thể hỗ trợ documentation generation trong software development.

Lab 5.2 - Fixing faulty implementation with GitHub Copilot

Mục tiêu

Mục tiêu đánh giá khả năng của GitHub Copilot trong việc hỗ trợ lập trình viên phát hiện và khắc phục lỗi trong quá trình phát triển phần mềm.

Luồng hoạt động



Code

Code có lỗi

```

from dataclasses import dataclass

from unittest import TestCase, main

class Singleton(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls.__name__] = super(Singleton, cls).__call__(*args,
**kwargs)

        return cls._instances[cls.__name__]

    @dataclass
class Environment(metaclass=Singleton):
    name: str = 'Production' # This is the default environment
    version: int = 1

class TestSingleton(TestCase):

    def test_singleton_should_return_same_instance(self):

        @dataclass
        class A(metaclass=Singleton):

```

```
a: int = 0
```

```
self.assertEqual(A(2), A())
```

```
if __name__ == "__main__":
```

```
    main()
```

Kết quả

```
(base) PS F:\university\hk4_2\SEMINAR\SGU2026-SEMINAR_CA_NHAN\Bài tập tuần 5\lab 5\labs> python lab52.py
F
=====
FAIL: test_singleton_should_return_same_instance (__main__.TestSingleton.test_singleton_should_return_same_instance)
-----
Traceback (most recent call last):
  File "F:\university\hk4_2\SEMINAR\SGU2026-SEMINAR_CA_NHAN\Bài tập tuần 5\lab 5\labs\lab52.py", line 27, in test_singleton_should_return_same_instance
    self.assertEqual(A(2), A())
AssertionError: TestSingleton.test_singleton_should_return_same_instance.<locals>.A(a=2) != TestSingleton.test_singleton_should_return_same_instance.<locals>.A(a=0)
-----
Ran 1 test in 0.001s

FAILED (failures=1)
```

Prompt với Github Copilot

@terminal /explain the failing unit test

Terminal

Used 1 reference

Based on the terminal output, I can see the failing unit test. Here's what's happening:

Failing Test: `test_singleton_should_return_same_instance`

Location: `lab52.py` at line 27

The Problem:

The test expects that calling `A(2)` and `A()` should return the **same instance**, but they're returning different instances with different values:

- `A(a=2)` - first call with argument 2
- `A(a=0)` - second call without argument

What's Failing:

```
self.assertEqual(A(2), A())  
AssertionError: TestSingleton.test_singleton_should_return_same_insta
```

The Fix

+  `lab52.py`

Describe what to build

+  Agent  Claude Haiku 4.5 



Trick bôi đen method *call* và dùng lệnh “ /fix cls instantiation” để fix

/fix cls instantiation

lab52.py:8-11

Plan

The issue is that the Singleton metaclass is storing instances by class name (`cls.__name__`), but when the same class `A` is defined in different test methods, they share the same name but are technically different classes. Additionally, the current implementation doesn't handle the case where arguments differ between calls.

Steps to fix:

1. Store instances by the actual class object (`cls`) instead of `cls.__name__` to properly distinguish different class definitions
2. This ensures each unique class definition gets its own singleton instance
3. The first instantiation `A(2)` will create and store the instance
4. The second call `A()` will return the cached instance (ignoring the default argument)

Solution

```
from dataclasses import dataclass
from unittest import TestCase, main
```

```
class Singleton(type):
```

+ lab52.py

Describe what to build

+ Agent Claude Haiku 4.5



```

class Singleton(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super(Singleton, cls).__call__(*arg
            return cls._instances[cls]

@dataclass
class Environment(metaclass=Singleton):
    name: str = 'Production' # This is the default environment
    version: int = 1

class TestSingleton(TestCase):

    def test_singleton_should_return_same_instance(self):

        @dataclass
        class A(metaclass=Singleton):
            a: int = 0

        self.assertEqual(A(2), A())

if __name__ == "__main__":
    main()

```

+  lab52.py

Describe what to build

+  Agent ▾ Claude Haiku 4.5 ▾ 



Code sau khi sửa

```
from dataclasses import dataclass

from unittest import TestCase, main

class Singleton(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super(Singleton, cls).__call__(*args, **kwargs)
        return cls._instances[cls]

@dataclass
class Environment(metaclass=Singleton):
    name: str = 'Production' # This is the default environment
    version: int = 1

class TestSingleton(TestCase):

    def test_singleton_should_return_same_instance(self):

        @dataclass
        class A(metaclass=Singleton):
```

```
a: int = 0

self.assertEqual(A(2), A())

if __name__ == "__main__":
    main()
```

Kết quả:

```
(base) PS F:\university\hk4_2\SEMINAR\SGU2026-SEMINAR_CA_NHAN\Bài tập tuần 5\lab 5\labs> python lab52.py
.
-----
Ran 1 test in 0.001s
OK
```

Phân tích

Trong lab này, GitHub Copilot được sử dụng để phát hiện và sửa lỗi trong implementation của Singleton pattern.

Lỗi nằm ở đoạn:

```
cls._instances[cls.__name__]
```

Việc sử dụng ”cls.__name__” làm key có thể gây lỗi vì:

- key không còn là class object
- logic singleton bị sai khi lookup instance

Copilot đề xuất sửa thành:

```
cls._instances[cls]
```

Sau khi sửa và test lại thì pass vì cả hai lời gọi constructor trả về cùng một instance.

3. Summary

Chương 5 tập trung vào việc ứng dụng Generative AI thông qua các công cụ và nền tảng tích hợp trong môi trường phát triển phần mềm, cụ thể là OpenAI API và GitHub Copilot. Chương này trình bày cách các mô hình AI có thể được sử dụng như một trợ lý lập trình để hỗ trợ nhiều nhiệm vụ trong quy trình phát triển phần mềm, bao gồm sinh mã nguồn, tạo tài liệu và sửa lỗi.

Thông qua các bài thí nghiệm, chương này minh họa cách OpenAI API có thể được sử dụng để tự động tạo docstring cho hàm Python, đồng thời cho thấy khả năng của GitHub Copilot trong việc hỗ trợ phát hiện và khắc phục lỗi triển khai trong mã nguồn. Kết quả cho thấy việc tích hợp Generative AI vào các công cụ phát triển phần mềm có thể giúp lập trình viên tăng năng suất, giảm thời gian viết tài liệu và cải thiện hiệu quả debugging.